



Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts

W3C Recommendation 26 June 2007

This version:

<http://www.w3.org/TR/2007/REC-wsdl20-adjuncts-20070626>

Latest version:

<http://www.w3.org/TR/wsdl20-adjuncts>

Previous version:

<http://www.w3.org/TR/2007/PR-wsdl20-adjuncts-20070523>

Editors:

Roberto Chinnici, Sun Microsystems
Hugo Haas, W3C
Amelia A. Lewis, TIBCO Software
Jean-Jacques Moreau, Canon
David Orchard, BEA Systems
Sanjiva Weerawarana, WSO2

Please refer to the [errata](#) for this document, which may include some normative corrections.

This document is also available in these non-normative formats: [PDF](#), [PostScript](#), [XML](#), and [plain text](#).

See also [translations](#).

[Copyright](#) © 2007 [W3C](#)[®] ([MIT](#), [ERCIM](#), [Keio](#)), All Rights Reserved. W3C [liability](#), [trademark](#) and [document use](#) rules apply.

Abstract

WSDL 2.0 is the Web Services Description Language, an XML language for describing Web services. This document, "Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts", specifies predefined extensions for use in WSDL 2.0:

- Message exchange patterns
- Operation safety
- Operation styles
- Binding extensions for SOAP and HTTP

Status of this Document

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the [W3C technical reports index](#) at <http://www.w3.org/TR/>.

This is the [W3C Recommendation](#) of Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts for review by W3C Members and other interested parties. It has been produced by the [Web Services Description Working Group](#), which is part of the [W3C Web Services Activity](#).

Please send comments about this document to the public public-ws-desc-comments@w3.org mailing list ([public archive](#)).

The Working Group released a test suite along with an [implementation report](#). A [diff-marked version against the previous version of this document](#) is available.

This document has been reviewed by W3C Members, by software developers, and by other W3C groups and interested parties, and is endorsed by the Director as a W3C Recommendation. It is a stable document and may be used as reference material or cited from another document. W3C's role in making the Recommendation is to draw attention to the specification and to promote its widespread deployment. This enhances the functionality and interoperability of the Web.

This document is governed by the [24 January 2002 CPP](#) as amended by the [W3C Patent Policy Transition Procedure](#). W3C maintains a [public list of any patent disclosures](#) made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes contains [Essential Claim\(s\)](#) must disclose the information in accordance with [section 6 of the W3C Patent Policy](#).

Table of Contents

1. [Introduction](#)
 - 1.1 [Notational Conventions](#)
 - 1.2 [Assertions](#)
2. [Predefined Message Exchange Patterns](#)
 - 2.1 [Template for Message Exchange Patterns](#)
 - 2.1.1 [Pattern Name](#)
 - 2.2 [Fault Propagation Rules](#)
 - 2.2.1 [Fault Replaces Message propagation rule](#)
 - 2.2.2 [Message Triggers Fault propagation rule](#)
 - 2.2.3 [No Faults propagation rule](#)
 - 2.3 [Message Exchange Patterns](#)
 - 2.3.1 [In-Only message exchange pattern](#)
 - 2.3.2 [Robust In-Only message exchange pattern](#)
 - 2.3.3 [In-Out message exchange pattern](#)
 - 2.4 [Security Considerations](#)
3. [Predefined Extensions](#)
 - 3.1 [Operation safety](#)
 - 3.1.1 [Relationship to WSDL Component Model](#)
 - 3.1.2 [XML Representation](#)
 - 3.1.3 [Mapping from XML Representation to Component Properties](#)
4. [Predefined Operation Styles](#)
 - 4.1 [RPC Style](#)
 - 4.1.1 [wrpc:signature Extension](#)
 - 4.1.2 [XML Representation of the wrpc:signature Extension](#)
 - 4.1.3 [wrpc:signature Extension Mapping To Properties of an Interface Operation component](#)
 - 4.2 [IRI Style](#)
 - 4.3 [Multipart style](#)
5. [WSDL SOAP Binding Extension](#)
 - 5.1 [SOAP Syntax Summary \(Non-Normative\)](#)
 - 5.2 [Identifying the use of the SOAP Binding](#)
 - 5.3 [SOAP Binding Rules](#)
 - 5.4 [Specifying the SOAP Version](#)
 - 5.4.1 [Description](#)
 - 5.4.2 [Relationship to WSDL Component Model](#)
 - 5.4.3 [XML Representation](#)
 - 5.4.4 [Mapping from XML Representation to Component properties](#)
 - 5.5 [Specifying the SOAP Underlying Protocol](#)
 - 5.5.1 [Description](#)
 - 5.5.2 [Relationship to WSDL Component Model](#)
 - 5.5.3 [XML Representation](#)

- 5.5.4 [Mapping from XML Representation to Component Properties](#)
- 5.6 [Binding Faults](#)
 - 5.6.1 [Description](#)
 - 5.6.2 [Relationship to WSDL Component Model](#)
 - 5.6.3 [XML Representation](#)
 - 5.6.4 [Mapping XML Representation to Component Properties](#)
- 5.7 [Binding Operations](#)
 - 5.7.1 [Description](#)
 - 5.7.2 [Relationship to WSDL Component Model](#)
 - 5.7.3 [XML Representation](#)
 - 5.7.4 [Mapping from XML Representation to Component Properties](#)
- 5.8 [Declaring SOAP Modules](#)
 - 5.8.1 [Description](#)
 - 5.8.2 [Relationship to WSDL Component Model](#)
 - 5.8.3 [SOAP Module component](#)
 - 5.8.4 [XML Representation](#)
 - 5.8.5 [Mapping from XML Representation to Component Properties](#)
 - 5.8.6 [IRI Identification Of A SOAP Module component](#)
- 5.9 [Declaring SOAP Header Blocks](#)
 - 5.9.1 [Description](#)
 - 5.9.2 [Relationship to WSDL Component Model](#)
 - 5.9.3 [SOAP Header Block component](#)
 - 5.9.4 [XML Representation](#)
 - 5.9.5 [Mapping XML Representation to Component Properties](#)
 - 5.9.6 [IRI Identification Of A SOAP Header Block component](#)
- 5.10 [WSDL SOAP 1.2 Binding](#)
 - 5.10.1 [Identifying a WSDL SOAP 1.2 Binding](#)
 - 5.10.2 [Description](#)
 - 5.10.3 [SOAP 1.2 Binding Rules](#)
 - 5.10.4 [Binding WSDL 2.0 MEPs to SOAP 1.2 MEPs](#)
 - 5.10.4.1 [WSDL In-Out to SOAP Request-Response](#)
 - 5.10.4.1.1 [The Client](#)
 - 5.10.4.1.2 [The Service](#)
 - 5.10.4.2 [WSDL In-Out to SOAP SOAP-Response](#)
 - 5.10.4.2.1 [The Client](#)
 - 5.10.4.2.2 [The Service](#)
 - 5.10.4.3 [WSDL In-Only to SOAP Request-Response](#)
 - 5.10.4.3.1 [The Client](#)
 - 5.10.4.3.2 [The Service](#)
 - 5.10.4.4 [WSDL Robust-In-Only to SOAP Request-Response](#)
 - 5.10.4.4.1 [The Client](#)
 - 5.10.4.4.2 [The Service](#)
- 5.11 [Conformance](#)
- 6. [WSDL HTTP Binding Extension](#)
 - 6.1 [Identifying the use of the HTTP Binding](#)
 - 6.2 [HTTP Syntax Summary \(Non-Normative\)](#)
 - 6.3 [Supported Extensions](#)
 - 6.4 [HTTP Binding Rules](#)
 - 6.4.1 [HTTP Method Selection](#)
 - 6.4.2 [HTTP Content Encoding Selection](#)
 - 6.4.3 [Payload Construction And Serialization Format](#)
 - 6.4.3.1 [Serialization rules for XML messages](#)
 - 6.4.4 [Default input and output serialization format](#)
 - 6.4.5 [HTTP Header Construction](#)
 - 6.4.6 [HTTP Request IRI](#)
 - 6.5 [Binding Operations](#)
 - 6.5.1 [Description](#)
 - 6.5.2 [Relationship to WSDL Component Model](#)
 - 6.5.3 [Specification of serialization rules allowed](#)
 - 6.5.4 [XML Representation](#)
 - 6.5.5 [Mapping from XML Representation to Component Properties](#)
 - 6.6 [Declaring HTTP Headers](#)

- 6.6.1 [Description](#)
- 6.6.2 [Relationship to WSDL Component Model](#)
- 6.6.3 [HTTP Header component](#)
- 6.6.4 [XML Representation](#)
- 6.6.5 [Mapping from XML Representation to Component Properties](#)
- 6.6.6 [IRI Identification Of An HTTP Header component](#)
- 6.7 [Specifying HTTP Error Code for Faults](#)
 - 6.7.1 [Description](#)
 - 6.7.2 [Relationship to WSDL Component Model](#)
 - 6.7.3 [XML Representation](#)
 - 6.7.4 [Mapping from XML Representation to Component Properties](#)
- 6.8 [Serialization Format of Instance Data](#)
 - 6.8.1 [Serialization of the instance data in parts of the HTTP request IRI](#)
 - 6.8.1.1 [Construction of the request IRI using the {http location}_property](#)
 - 6.8.2 [Serialization as application/x-www-form-urlencoded](#)
 - 6.8.2.1 [Case of elements cited in the {http location}_property](#)
 - 6.8.2.2 [Serialization of content of the instance data not cited in the {http location}_property](#)
 - 6.8.2.2.1 [Construction of the query string](#)
 - 6.8.2.2.2 [Controlling the serialization of the query string in the request IRI](#)
 - 6.8.2.2.3 [Serialization in the request IRI](#)
 - 6.8.2.2.4 [Serialization in the message body](#)
 - 6.8.3 [Serialization as application/xml](#)
 - 6.8.4 [Serialization as multipart/form-data](#)
- 6.9 [Specifying the Content Encoding](#)
 - 6.9.1 [Description](#)
 - 6.9.2 [Relationship to WSDL Component Model](#)
 - 6.9.3 [XML Representation](#)
 - 6.9.4 [Mapping from XML Representation to Component Properties](#)
- 6.10 [Specifying the Use of HTTP Cookies](#)
 - 6.10.1 [Description](#)
 - 6.10.2 [Relationship to WSDL Component Model](#)
 - 6.10.3 [XML Representation](#)
 - 6.10.4 [Mapping from XML Representation to Component Properties](#)
- 6.11 [Specifying HTTP Access Authentication](#)
 - 6.11.1 [Description](#)
 - 6.11.2 [Relationship to WSDL Component Model](#)
 - 6.11.3 [XML Representation](#)
 - 6.11.4 [Mapping from XML Representation to Component Properties](#)
- 6.12 [Conformance](#)
- 7. [References](#)
 - 7.1 [Normative References](#)
 - 7.2 [Informative References](#)

Appendices

- A. [Acknowledgements](#) (Non-Normative)
- B. [Component Summary](#) (Non-Normative)
- C. [Assertion Summary](#) (Non-Normative)

1. Introduction

The Web Services Description Language Version 2.0 (WSDL 2.0) [[WSDL 2.0 Core Language](#)] provides a model and an XML format for describing Web services. WSDL 2.0 enables one to separate the description of the abstract functionality offered by a service from concrete details of a service description such as "how" and "where" that functionality is offered.

This document, "Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts", specifies predefined extensions for use in WSDL 2.0:

- Message exchange patterns: [2. Predefined Message Exchange Patterns](#)

- Operation safety declaration: [3. Predefined Extensions](#)
- Operation styles: [4. Predefined Operation Styles](#)
- Binding extensions:
 - A SOAP 1.2 [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)] binding extension: [5. WSDL SOAP Binding Extension](#)
 - An HTTP/1.1 [[IETF RFC 2616](#)] binding extension: [6. WSDL HTTP Binding Extension](#)

This document depends on "Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language" [[WSDL 2.0 Core Language](#)]. See also the "Web Services Description Language (WSDL) Version 2.0 Part 0: Primer" [[WSDL 2.0 Primer](#)] for more information and examples.

1.1 Notational Conventions

The keywords "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC2119 [[IETF RFC 2119](#)].

This specification uses a number of namespace prefixes throughout; they are listed in [Table 1-1](#). Note that the choice of any namespace prefix is arbitrary and not semantically significant (see [[XML Information Set](#)]).

Table 1-1. Prefixes and Namespaces used in this specification

Prefix	Namespace	Notes
wsdl	"http://www.w3.org/ns/wsdl"	This namespace is defined in [WSDL 2.0 Core Language]. A normative XML Schema [XML Schema Structures], [XML Schema Datatypes] document for the "http://www.w3.org/ns/wsdl" namespace can be found at http://www.w3.org/ns/wsdl . This namespace is used as the default namespace throughout this specification.
wsdlx	"http://www.w3.org/ns/wsdl-extensions"	This specification extends in section 3. Predefined Extensions the "http://www.w3.org/ns/wsdl-extensions" namespace defined in [WSDL 2.0 Core Language]. A normative XML Schema [XML Schema Structures], [XML Schema Datatypes] document for the "http://www.w3.org/ns/wsdl-extensions" namespace can be found at http://www.w3.org/ns/wsdl-extensions .
wsoap	"http://www.w3.org/ns/wsdl/soap"	Defined by this specification. A normative XML Schema [XML Schema Structures], [XML Schema Datatypes] document for the "http://www.w3.org/ns/wsdl/soap" namespace can be found at http://www.w3.org/ns/wsdl/soap .
whhttp	"http://www.w3.org/ns/wsdl/http"	Defined by this specification. A normative XML Schema [XML Schema Structures], [XML Schema Datatypes] document for the "http://www.w3.org/ns/wsdl/http" namespace can be found at http://www.w3.org/ns/wsdl/http .
wrpc	"http://www.w3.org/ns/wsdl/rpc"	Defined by this specification. A normative XML Schema [XML Schema Structures], [XML Schema Datatypes] document for the "http://www.w3.org/ns/wsdl/rpc" namespace can be found at http://www.w3.org/ns/wsdl/rpc .
xs	"http://www.w3.org/2001/XMLSchema"	Defined in the W3C XML Schema specification

	[XML Schema Structures] , [XML Schema Datatypes] .
--	--

Namespace names of the general form "http://example.org/..." and "http://example.com/..." represent application or context-dependent URIs [[IETF RFC 3986](#)].

All parts of this specification are normative, with the EXCEPTION of pseudo-schemas, examples, and sections explicitly marked as "Non-Normative". Pseudo-schemas are provided for each component, before the description of this component. They provide visual help for the XML [[XML 1.0](#)] serialization. The [syntax of BNF pseudo-schemas](#) is the same as the one used in [[WSDL 2.0 Core Language](#)].

1.2 Assertions

Assertions about WSDL 2.0 documents and components that are not enforced by the normative XML schema for WSDL 2.0 are marked by a dagger symbol (†) at the end of a sentence. Each assertion has been assigned a unique identifier that consists of a descriptive textual prefix and a unique numeric suffix. The numeric suffixes are assigned sequentially and never reused so there may be gaps in the sequence. The assertion identifiers MAY be used by implementations of this specification for any purpose, e.g. error reporting.

The assertions and their identifiers are summarized in section [C. Assertion Summary](#).

2. Predefined Message Exchange Patterns

Web Services Description Language (WSDL) message exchange patterns (hereafter simply 'patterns') define the sequence and cardinality of abstract messages listed in an operation. Message exchange patterns also define which other nodes send messages to, and receive messages from, the service implementing the operation.

A *node* is an agent ([section 2.3.2.2 Agent of the Web Services Architecture](#) [[Web Services Architecture](#)]) that can transmit and/or receive message(s) described in WSDL description(s) and process them.

Note:

A node MAY be accessible via more than one physical address or transport.[†]

WSDL message exchange patterns describe the interaction at the abstract (interface) level, which may be distinct from the pattern used by the underlying protocol binding (e.g. SOAP Message Exchange Patterns; section [5.10.3 SOAP 1.2 Binding Rules](#) contains the binding rules for the selection of a SOAP 1.2 message exchange pattern, based on the WSDL message exchange pattern in use for the SOAP binding extension defined in section [5. WSDL SOAP Binding Extension](#)).

By design, WSDL message exchange patterns abstract out specific message types. Patterns identify placeholders for messages, and placeholders are associated with specific message types by the operation using the pattern.

Unless explicitly stated otherwise, WSDL message exchange patterns also abstract out binding-specific information such as timing between messages, whether the pattern is synchronous or asynchronous, and whether the messages are sent over a single or multiple channels.

Like interfaces and operations, WSDL message exchange patterns do not exhaustively describe the set of messages exchanged between a service and other nodes; by some prior agreement, another node and/or the service MAY send messages (to each other or to other nodes) that are not described by the pattern.[†] For instance, even though a pattern can define a single message sent from a service to one other node, the Web service can in practice multicast that message to other nodes.

To maximize reuse, WSDL message exchange patterns identify a minimal contract between other parties and Web services, and contain only information that is relevant to both the Web service and another party.

This specification defines several message exchange patterns for use with *WSDL Version 2.0 Part 1: Core Language* [[WSDL 2.0 Core Language](#)]. Additional, non-normative patterns are available in [[WSDL 2.0 Additional MEPS](#)].

2.1 Template for Message Exchange Patterns

New message exchange patterns may be defined by any organization able and willing to do so. It is recommended that the patterns use the general template provided in [2.1.1 Pattern Name](#), after examination of existing predefined patterns.

2.1.1 Pattern Name

This pattern consists of [number] message[s, in order] as follows:

[enumeration, specifying, for each message] A[n optional] message:

1. indicated by an [Interface Message Reference](#) component whose {[message label](#)} is "[label]" and {[direction](#)} is "[direction]"
2. [received from|sent to] ['some' if first mention] node [node identifier]

This pattern uses the rule [fault ruleset reference].

An [Interface Operation](#) using this message exchange pattern has a {[message exchange pattern](#)} property with the value "[pattern IRI]".

Note: In the template, the bracketed items indicate a replacement operation. Substitute the correct terms for each bracketed item.

Note: the "received from" and "sent to" are always from the point of view of the service, and participating nodes other than the service are implicitly identified as the originators of or destinations for messages in the exchange.

2.2 Fault Propagation Rules

WSDL patterns specify their fault propagation model using standard rulesets to indicate where faults can occur. The most common patterns for fault propagation are defined in the following subsections, and referenced by the patterns in [2.3 Message Exchange Patterns](#).

"Propagation" is defined as a best-effort attempt to transmit the fault message to its designated recipient.

WSDL patterns specify propagation of faults, not their generation. Nodes that generate faults MUST attempt to propagate the faults in accordance with the governing ruleset, but it is understood that any delivery of a network message is best effort, not guaranteed.[‡] The rulesets establish the direction of the fault message and the fault recipient; they do not provide reliability or other delivery guarantees. When a fault is generated, the generating node MUST attempt to propagate the fault, and MUST do so in the direction and to the recipient specified by the ruleset.[‡] However, extensions or binding extensions MAY modify these rulesets.[‡] For example, WS-Addressing [[WSA 1.0 Core](#)] defines a "FaultTo" address for messages, which is used in lieu of the recipient nominated by the ruleset.

Generation of a fault, regardless of ruleset, terminates the exchange.[‡]

Binding extensions, features, or extension specifications can override the semantics of a fault propagation ruleset, but this practice is strongly discouraged.

2.2.1 Fault Replaces Message propagation rule

When the Fault Replaces Message propagation rule is in effect, any message after the first in the pattern MAY be replaced with a fault message, which MUST have identical direction.[‡] The fault message MUST be delivered to the same target node as the message it replaces, unless otherwise specified by an extension or binding extension. If there is no path to this node, the fault MUST be discarded.[‡]

The Fault Replaces Message propagation rule is identified by the following URI:

<http://www.w3.org/ns/wsd1/fault-replaces-message>

2.2.2 Message Triggers Fault propagation rule

When the Message Triggers Fault propagation rule is in effect, any message, including the first in the pattern, MAY trigger a fault message, which MUST have opposite direction. [‡] The fault message MUST be delivered to the originator of the triggering message, unless otherwise specified by an extension or binding extension. Any node MAY propagate a fault message, and MUST NOT do so more than once for each triggering message. If there is no path to the originator, the fault MUST be discarded. [‡]

The Message Triggers Fault propagation rule is identified by the following URI:

<http://www.w3.org/ns/wsd1/message-triggers-fault>

2.2.3 No Faults propagation rule

When the No Faults propagation rule is in effect, faults MUST NOT be propagated. [‡]

The No Faults propagation rule is identified by the following URI: <http://www.w3.org/ns/wsd1/no-faults>

2.3 Message Exchange Patterns

WSDL patterns are described in terms of the WSDL component model, specifically the [Interface Message Reference](#) and [Interface Fault Reference](#) components.

2.3.1 In-Only message exchange pattern

The `in-only` message exchange pattern consists of exactly one message as follows: [‡]

1. A message:
 - indicated by a [Interface Message Reference](#) component whose `{message label}` is "In" and `{direction}` is "in"
 - received from some node N

The `in-only` message exchange pattern uses the rule [2.2.3 No Faults propagation rule](#). [‡]

An operation using this message exchange pattern has a `{message exchange pattern}` property with the value "<http://www.w3.org/ns/wsd1/in-only>".

2.3.2 Robust In-Only message exchange pattern

The `robust-in-only` message exchange pattern consists of exactly one message as follows: [‡]

1. A message:
 - indicated by a [Interface Message Reference](#) component whose `{message label}` is "In" and `{direction}` is "in"
 - received from some node N

The `robust in-only` message exchange pattern uses the rule [2.2.2 Message Triggers Fault propagation rule](#). [‡]

An operation using this message exchange pattern has a `{message exchange pattern}` property with the value "<http://www.w3.org/ns/wsd1/robust-in-only>".

2.3.3 In-Out message exchange pattern

The `in-out` message exchange pattern consists of exactly two messages, in order, as follows: [‡]

1. A message:
 - indicated by a [Interface Message Reference](#) component whose `{message label}` is "In" and `{direction}` is "in"
 - received from some node N
2. A message:

- indicated by a [Interface Message Reference](#) component whose `{message label}` is "Out" and `{direction}` is "out"
- sent to node N

The in-out message exchange pattern uses the rule [2.2.1 Fault Replaces Message propagation rule](#).[‡]

An operation using this message exchange pattern has a `{message exchange pattern}` property with the value "http://www.w3.org/ns/wSDL/in-out".

2.4 Security Considerations

Note that many of the message exchange patterns defined above describe responses to an initial message (either a normal response message or a fault.)

Such responses can be used in attempts to disrupt, attack, or map a network, host, or services. When such responses are directed to an address other than that originating the initial message, the source of an attack can be obscured, or blame laid on a third party, or denial-of-service attacks can be enabled or exacerbated.

Security mechanisms addressing such attacks can prevent the delivery of response messages to the receiving node. Conformance to the message exchange pattern is measured prior to the application of these security mechanisms.

3. Predefined Extensions

3.1 Operation safety

This section defines an extension to WSDL 2.0 [[WSDL 2.0 Core Language](#)] that allows marking an operation as a safe interaction, as defined in [section 3.4. Safe Interactions](#) of [[Web Architecture](#)].

This extension MAY be used for setting defaults in bindings, such as in the HTTP binding (see [6.5.5 Mapping from XML Representation to Component Properties](#)).

3.1.1 Relationship to WSDL Component Model

The safety extension adds the following property to the [Interface Operation](#) component model (defined in [[WSDL 2.0 Core Language](#)]):

- `{safe}` REQUIRED. An `xs:boolean` indicating whether the operation is asserted to be safe for users to invoke. If this property is "false", then no assertion has been made about the safety of the operation, thus the operation MAY or MAY NOT be safe. However, an operation SHOULD be marked safe if it meets the criteria for a safe interaction defined in Section 3.4 of [[Web Architecture](#)].[‡]

3.1.2 XML Representation

```
<description>
  <interface>
    <operation name="xs:NCName" pattern="xs:anyURI"
      wsd1x:safe="xs:boolean"? >
    </operation>
  </interface>
</description>
```

The XML representation for the safety extension is an *attribute information item* with the following Infoset properties:

- An OPTIONAL *safe attribute information item* with the following Infoset properties:[‡]
 - A [local name] of `safe`
 - A [namespace name] of "http://www.w3.org/ns/wSDL-extensions"

- A type of *xs:boolean*

3.1.3 Mapping from XML Representation to Component Properties

See [Table 3-1](#).

Table 3-1. Mapping from XML Representation to Interface Operation component Extension Properties

Property	Value
{ safe }	The actual value of the <i>safe attribute information item</i> , if present; otherwise the value "false".

4. Predefined Operation Styles

This section defines [operation styles](#) that can be used to place constraints on [Interface Operation](#) components, in particular with respect to the format of the messages they refer to. The serialization formats defined in section [6.8 Serialization Format of Instance Data](#) require bound [Interface Operation](#) components to have one or more of the styles defined in this section.

4.1 RPC Style

The RPC style is selected by including the value "http://www.w3.org/ns/wsdli/style/rpc" in the {[style](#)} property of an [Interface Operation](#) component.

An [Interface Operation](#) component conforming to the RPC style MUST obey the constraints listed further below. Also, if the *wrpc:signature* extension is engaged simultaneously, the corresponding *attribute information item* MUST be valid according to the schema for the extension and additionally MUST obey the constraints listed in [4.1.1 wrpc:signature Extension](#) and [4.1.2 XML Representation of the wrpc:signature Extension](#).

Furthermore, the associated messages MUST conform to the rules below, described using XML Schema [[XML Schema Structures](#)]. Note that operations containing messages described by other type systems may also indicate use of the RPC style, as long as they are constructed in such a way as to follow these rules.

If the RPC style is used by an [Interface Operation](#) component then its {[message exchange pattern](#)} property MUST have the value either "http://www.w3.org/ns/wsdli/in-only" or "http://www.w3.org/ns/wsdli/in-out".[‡]

If the [Interface Operation](#) component uses a {[message exchange pattern](#)} for which there is no output element, i.e. "http://www.w3.org/ns/wsdli/in-only", then the conditions stated below that refer to output elements MUST be considered to be implicitly satisfied.

- The value of the {[message content model](#)} property for the [Interface Message Reference](#) components of the {[interface message references](#)} property MUST be "#element".[‡]
- The content model of input and output {[element declaration](#)} elements MUST be defined using a complex type that contains a sequence from XML Schema.[‡]
- The input sequence MUST only contain elements and element wildcards.[‡] It MUST NOT contain other structures such as *xs:choice*. The input sequence MUST NOT contain more than one element wildcard.[‡] The element wildcard, if present, MUST appear after any elements.[‡]
- The output sequence MUST only contain elements.[‡] It MUST NOT contain other structures such as *xs:choice*.
- Both the input and output sequences MUST contain only local element children.[‡] Note that these child elements MAY contain the following attributes: *nillable*, *minOccurs* and *maxOccurs*.
- The local name of input element's QName MUST be the same as the [Interface Operation](#) component's name.[‡]

- Input and output elements MUST both be in the same namespace.[‡]
- The complex type that defines the body of an input or an output element MUST NOT contain any local attributes.[‡] Extension attributes are allowed for purposes of managing the message infrastructure (e.g. adding identifiers to facilitate digitally signing the contents of the message). They must not be considered as part of the application data that is conveyed by the message. Therefore, they are never included in an RPC signature (see [4.1.1 wrpc:signature Extension](#)).
- If elements with the same qualified name appear as children of both the input and output elements, then they MUST both be declared using the same named type.[‡]
- The input or output sequence MUST NOT contain multiple children elements declared with the same name.[‡]

4.1.1 wrpc:signature Extension

The `wrpc:signature` extension *attribute information item* MAY be used in conjunction with the RPC style to describe the exact signature of the function represented by an operation that uses the RPC style.

When present, the `wrpc:signature` extension contributes the following property to the [Interface Operation](#) component it is applied to:

- {rpc signature} OPTIONAL, but MUST be present when the style is RPC[‡]. A list of pairs (q, t) whose first component is of type `xs:QName` and whose second component is of type `xs:token`. Values for the second component MUST be chosen among the following four: `"#in"`, `"#out"`, `"#inout"` `"#return"`.[‡]

The value of the {[rpc signature](#)} property MUST satisfy the following conditions:

- The value of the first component of each pair (q, t) MUST be unique within the list.[‡]
- For each child element of the input and output messages of the operation, a pair (q, t) , whose first component q is equal to the qualified name of that element, MUST be present in the list, with the caveat that elements that appear with cardinality greater than one MUST be treated as a single element.[‡]
- For each pair $(q, \text{#in})$, there MUST be a child element of the input element with a name of q . There MUST NOT be a child element of the output element with the name of q .[‡]
- For each pair $(q, \text{#out})$, there MUST be a child element of the output element with a name of q . There MUST NOT be a child element of the input element with the name of q .[‡]
- For each pair $(q, \text{#inout})$, there MUST be a child element of the input element with a name of q . There MUST also be a child element of the output element with the name of q .[‡]
- For each pair $(q, \text{#return})$, there MUST be a child element of the output element with a name of q . There MUST NOT be a child element of the input element with the name of q .[‡]

The function signature defined by a `wrpc:signature` extension is determined as follows:

1. Start with the value of the {[rpc signature](#)} property, a (possibly empty) list of pairs of this form:

$[(q_0, t_0), (q_1, t_1), \dots]$

2. Filter the elements of this list into two lists, the first one (L_1) comprising pairs whose t component is one of `{#in, #out, #inout}`, the second (L_2) pairs whose t component is `#return`. During the composition of L_1 and L_2 , the relative order of members in the original list MUST be preserved.

For ease of visualization, let's denote the two lists as:

(L_1) $[(a_0, u_0), (a_1, u_1), \dots]$

and

(L_2) $[(r_0, \text{#return}), (r_1, \text{#return}), \dots]$

respectively.

3. Then, if the input sequence ends with an element wildcard, the formal signature of the function is:

$$f([d0] a0, [d1] a1, \dots, rest) \Rightarrow (r0, r1, \dots)$$

where *rest* is a formal parameter representing the elements in the input message matched by the element wildcard.

Otherwise the formal signature of the function is:

$$f([d0] a0, [d1] a1, \dots) \Rightarrow (r0, r1, \dots)$$

i.e.:

- the list of formal arguments to the function is $[a0, a1, \dots]$;
- the direction *d* of each formal argument *a* is one of $[in]$, $[out]$, $[inout]$, determined according to the value of its corresponding *u* token;
- the list of formal return parameters of the function is $[r0, r1, \dots]$;
- each formal argument and formal return parameter is typed according to the type of the child element identified by it (unique per the conditions given above).

Note:

The `wrpc:signature` extension allows the specification of multiple return values for an operation. Several popular programming languages support multiple return values for a function. Moreover, for languages which do not, the burden on implementers should be small, as typically multiple return values will be mapped to a single return value of a structure type (or its closest language-specific equivalent).

4.1.2 XML Representation of the `wrpc:signature` Extension

The XML representation for the RPC signature extension is an *attribute information item* with the following Infoset properties:

- A [local name] of `signature`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/rpc"`

The type of the `signature` *attribute information item* is a list type whose item type is the union of the `xs:QName` type and the subtype of the `xs:token` type restricted to the following four values: `"#in"`, `"#out"`, `"#inout"`, `"#return"`. See [Example 4-1](#) for an excerpt from the normative schema definition of this type.

Additionally, each even-numbered item (0, 2, 4, ...) in the list MUST be of type `xs:QName` and each odd-numbered item (1, 3, 5, ...) in the list MUST be of the subtype of `xs:token` described in the previous paragraph.[‡]

Example 4-1. Definition of the `wrpc:signature` extension

```
<xs:attribute name="signature" type="wrpc:signatureType"/>

<xs:simpleType name="signatureType">
  <xs:list itemType="wrpc:signatureItemType"/>
</xs:simpleType>

<xs:simpleType name="signatureItemType">
  <xs:union memberTypes="xs:QName wrpc:directionToken"/>
</xs:simpleType>

<xs:simpleType name="directionToken">
  <xs:restriction base="xs:token">
    <xs:enumeration value="#in"/>
    <xs:enumeration value="#out"/>
    <xs:enumeration value="#inout"/>
    <xs:enumeration value="#return"/>
  </xs:restriction>
</xs:simpleType>
```

4.1.3 `wrpc:signature` Extension Mapping To Properties of an Interface Operation component

A `wrpc:signature` extension *attribute information item* is mapped to the following property of the [Interface Operation](#) component defined by its [owner].

Table 4-1. Mapping of a `wrpc:signature` Extension to Interface Operation component Properties

Property	Value
<code>{rpc signature}</code>	A list of (<code>xs:QName</code> , <code>xs:token</code>) pairs formed by grouping the items present in the actual value of the <code>wrpc:signature</code> <i>attribute information item</i> in the order in which they appear there.

4.2 IRI Style

The IRI style is selected by including the value "<http://www.w3.org/ns/wsdli/style/iri>" in the `{style}` property of an [Interface Operation](#) component.

When using this style, the value of the `{message content model}` property of the [Interface Message Reference](#) component corresponding to the initial message of the message exchange pattern MUST be "#element".[‡]

Use of this value indicates that XML Schema [[XML Schema Structures](#)] was used to define the schema of the `{element declaration}` property of the [Interface Message Reference](#) component of the [Interface Operation](#) component corresponding to the initial message of the message exchange pattern. This schema MUST adhere to the rules below:

- The content model of this element is defined using a complex type that contains a sequence from XML Schema.
- The sequence MUST only contain elements.[‡] It MUST NOT contain other structures such as `xs:choice`. There are no occurrence constraints on the sequence.
- The sequence MUST contain only local element children.[‡] Note these child elements can contain the `nillable` attribute.
- The localPart of the element's QName MUST be the same as the [Interface Operation](#) component's `{name}`.[‡]
- The complex type that defines the body of the element or its children elements MUST NOT contain any attributes.[‡]
- The children elements of the sequence MUST derive from `xs:simpleType`, and MUST NOT be of the type or derive from `xs:QName`, `xs:NOTATION`, `xs:hexBinary` OR `xs:base64Binary`.[‡]

4.3 Multipart style

The Multipart style is selected by including the value "<http://www.w3.org/ns/wsdli/style/multipart>" in the `{style}` property of an [Interface Operation](#) component.

When using this style, the value of the `{message content model}` property of the [Interface Message Reference](#) component corresponding to the initial message of the message exchange pattern MUST be "#element".[‡]

Use of this value indicates that XML Schema [[XML Schema Structures](#)] was used to define the schema of the `{element declaration}` property of the [Interface Message Reference](#) component of the [Interface Operation](#) component corresponding to the initial message of the message exchange pattern. This schema MUST adhere to the rules below:

- The content model of this element is defined using a complex type that contains a sequence from XML Schema.
- The sequence MUST only contain elements.[‡] It MUST NOT contain other structures such as `xs:choice`.

- The sequence MUST contain only local element children.[‡] The attributes `minOccurs` and `maxOccurs` for these child elements MUST have a value 1.[‡] Note these child elements can contain the `nillable` attribute.
- The localPart of the element's QName MUST be the same as the [Interface Operation](#) component's `{name}`.[‡]
- The complex type that defines the body of the element or its children elements MUST NOT contain any attributes.[‡]
- The sequence MUST NOT contain multiple children element declared with the same local name.[‡]

5. WSDL SOAP Binding Extension

The SOAP binding extension described in this section is an extension for [\[WSDL 2.0 Core Language\]](#) to enable Web services applications to use SOAP. This binding extension is SOAP version independent ("1.2" as well as other versions) and extends WSDL 2.0 by adding properties to the [Binding](#) component, and its related components, as defined in [\[WSDL 2.0 Core Language\]](#). In addition, an XML Infoset representation for these additional properties is provided, along with a mapping from that representation to the various component properties.

As allowed in [\[WSDL 2.0 Core Language\]](#), a [Binding](#) component can exist without indicating a specific [Interface](#) component that it applies to. In this case, no [Binding Operation](#) or [Binding Fault](#) component can be present in the [Binding](#) component.

The SOAP binding extension is designed with the objective of minimizing what needs to be explicitly declared for common cases. This is achieved by defining a set of default rules that affect all [Interface Operation](#) components of an [Interface](#) component to which the SOAP binding extension is applied, unless specifically overridden by a [Binding Operation](#) component. Thus, if a given [Interface Operation](#) component is not referred to specifically by a [Binding Operation](#) component, then all the default rules apply to that [Interface Operation](#) component. As a result, in accordance with the requirements of [\[WSDL 2.0 Core Language\]](#), all operations of an [Interface](#) component will be bound by this binding extension.

Note: As in other parts of this specification, one could have done away with "default" properties at the component model level, and have set the value for the corresponding non-default properties in the XML mapping section. However, default properties are required for interface-less binding. Indeed, an interface-less binding has no means to set the non-default version of the property at the operation-level, since there is precisely no operation (there is not even an interface). Hence the mapping needs to be done elsewhere.

A subset of the HTTP properties specified in the HTTP binding extension defined in section [6. WSDL HTTP Binding Extension](#) are present in a SOAP binding when the SOAP binding uses HTTP as the underlying protocol, for example, when the value of the `{soap underlying protocol}` property of the [Binding](#) component is "http://www.w3.org/2003/05/soap/bindings/HTTP/". These properties MUST NOT be used unless the underlying protocol is HTTP.[‡] The allowed properties are the ones that describe the underlying protocol (HTTP):

- `{http location}` and `{http location ignore uncited}` on [Binding Operation](#) components, as defined in [6.5 Binding Operations](#) and [6.8.2.2 Controlling the serialization of the query string in the request IRI](#), respectively.
- `{http headers}` on [Binding Message Reference](#) and [Binding Fault](#) components, as defined in [6.6 Declaring HTTP Headers](#)
- `{http query parameter separator default}` on [Binding](#) components, `{http query parameter separator}` on [Binding Operation](#) components, as defined in [6.5.2 Relationship to WSDL Component Model](#)
- `{http content encoding default}` on [Binding](#) and [Binding Operation](#) components, `{http content encoding}` on [Binding Message Reference](#) and [Binding Fault](#) components, as defined in [6.9 Specifying the Content Encoding](#)
- `{http cookies}` on [Binding](#) components, as defined in [6.10 Specifying the Use of HTTP Cookies](#).
- `{http authentication scheme}` and `{http authentication realm}` on [Endpoint](#) components, as defined in [6.11 Specifying HTTP Access Authentication](#)

5.1 SOAP Syntax Summary (Non-Normative)

```

<description>
  <binding name="xs:NCName" interface="xs:QName"?
    type="http://www.w3.org/ns/wsdl/soap"
    whttp:queryParameterSeparatorDefault="xs:string"?
    whttp:contentEncodingDefault="xs:string"?
    whttp:cookies="xs:boolean"?
    wsoap:version="xs:string"?
    wsoap:protocol="xs:anyURI"
    wsoap:mepDefault="xs:anyURI"? >
  <documentation />*

  <wsoap:module ref="xs:anyURI" required="xs:boolean"? >
    <documentation />*
  </wsoap:module>*

  <fault ref="xs:QName"
    wsoap:code="union of xs:QName, xs:token"?
    wsoap:subcodes="union of (list of xs:QName), xs:token"?
    whttp:contentEncoding="xs:string"? >

    <documentation />*

    <wsoap:module ... />*
    <wsoap:header element="xs:QName" mustUnderstand="xs:boolean"?
      required="xs:boolean"? >
      <documentation />*
    </wsoap:header>*
    <whttp:header ... />*??

  </fault>*

  <operation ref="xs:QName"
    whttp:location="xs:anyURI"?
    whttp:contentEncodingDefault="xs:string"?
    whttp:queryParameterSeparator="xs:string"?
    whttp:ignoreUncited="xs:boolean"?
    wsoap:mep="xs:anyURI"?
    wsoap:action="xs:anyURI"? >

    <documentation />*

    <wsoap:module ... />*

    <input messageLabel="xs:NCName"?
      whttp:contentEncoding="xs:string"? >
      <documentation />*
      <wsoap:module ... />*
      <wsoap:header ... />*
      <whttp:header ... />*??
    </input>*

    <output messageLabel="xs:NCName"?
      whttp:contentEncoding="xs:string"? >
      <documentation />*
      <wsoap:module ... />*
      <wsoap:header ... />*
      <whttp:header ... />*??
    </output>*

    <infault ref="xs:QName"
      messageLabel="xs:NCName"? >
      <documentation />*
      <wsoap:module ... />*
    </infault>*

    <outfault ref="xs:QName"
      messageLabel="xs:NCName"? >
      <documentation />*
      <wsoap:module ... />*
    </outfault>*

```

```

    </operation>*

</binding>

<service>
  <endpoint name="xs:NCName" binding="xs:QName" address="xs:anyURI"?
    http:authenticationScheme="xs:token"??
    http:authenticationRealm="xs:string"?? >
    <documentation />*
  </endpoint>
</service>
</description>

```

Note:

The double question marks ("??") after the attributes in the `http` namespace indicates that those optional attributes only make sense when the SOAP binding uses HTTP as the underlying protocol, for example, when the value of the `wsoap:protocol` attribute is `"http://www.w3.org/2003/05/soap/bindings/HTTP/"`.

5.2 Identifying the use of the SOAP Binding

A [Binding](#) component (defined in [[WSDL 2.0 Core Language](#)]) is identified as a SOAP binding by assigning the value `"http://www.w3.org/ns/wsd/soap"` to the `{type}` property of the [Binding](#) component.

5.3 SOAP Binding Rules

- *Payload Construction.* When formulating the SOAP envelope to be transmitted, the contents of the payload (i.e., the contents of the SOAP Body *element information item* of the SOAP envelope) MUST be what is defined by the corresponding [Interface Message Reference](#) component.[‡] This is further subject to optimization by a feature in use which affects serialization, such as MTOM [[SOAP Message Transmission Optimization Mechanism](#)]. The following binding rules MUST be adhered to:
 - If the value of the `{message content model}` property of the [Interface Message Reference](#) component is `"#any"`, then the payload MAY be any one XML element.
 - If the value is `"#none"`, then the payload MUST be empty.[‡]
 - If the value is `"#element"`, then the payload MUST be the *element information item* identified by the `{element declaration}` property of the [Interface Message Reference](#) component.[‡]
 - If the [Interface Message Reference](#) component is declared using a non-XML type system (as considered in the Types section of [[WSDL 2.0 Core Language](#)]), then additional binding rules MUST be defined to indicate how to map those components into the SOAP envelope.[‡]

Note:

This SOAP binding extension only allows one single element in the SOAP body.

- *SOAP Header Construction.* If the `{soap headers}` property as defined in section [5.9 Declaring SOAP Header Blocks](#) exists and is not empty in a [Binding Message Reference](#) or [Binding Fault](#) component, then an *element information item* conforming to the element declaration of a [SOAP Header Block](#) component's `{element declaration}` property, in the `{soap headers}` property, MAY be turned into a SOAP header block for the corresponding message.

If the value of the [SOAP Header Block](#) component's `{required}` property is `"true"`, the inclusion of this SOAP header block is REQUIRED, otherwise it is OPTIONAL.

And, if the [SOAP Header Block](#) component's `{mustUnderstand}` property is present and its value is `"true"`, that particular SOAP header block MUST be marked with a `mustUnderstand` *attribute information item* with a value of `"true"` or `"1"` as per the SOAP specification.

SOAP header blocks other than the ones declared in the `{soap headers}` property may be present at run-time, such as the SOAP header blocks resulting from SOAP modules declared as explained in section [5.8 Declaring SOAP Modules](#).

5.4 Specifying the SOAP Version

5.4.1 Description

Every SOAP binding **MUST** indicate what version of SOAP is in use for the operations of the interface that this binding applies to.[‡]
By default, SOAP 1.2 [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)] is used.

5.4.2 Relationship to WSDL Component Model

The SOAP protocol specification adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {soap version} **REQUIRED**. A *xs:string*, to the [Binding](#) component.

5.4.3 XML Representation

```
<description>
  <binding name="xs:NCName" interface="xs:QName"? type="xs:anyURI"
    wsoap:version="xs:string"? >
    ...
  </binding>
</description>
```

The XML representation for specifying the SOAP version is an optional *attribute information item* with the following Infoset properties:

- A [local name] of `version`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/soap"`
- A type of *xs:string*

5.4.4 Mapping from XML Representation to Component properties

See [Table 5-1](#).

Table 5-1. Mapping from XML Representation to Binding component Extension Properties

Property	Value
{ soap version }	The actual value of the <code>wsoap:version</code> <i>attribute information item</i> , if present; otherwise "1.2".

5.5 Specifying the SOAP Underlying Protocol

5.5.1 Description

Every SOAP binding **MUST** indicate what underlying protocol is in use.[‡]

5.5.2 Relationship to WSDL Component Model

The SOAP protocol specification adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {soap underlying protocol} **REQUIRED**. A *xs:anyURI*, which is an absolute IRI as defined by [[IETF RFC 3987](#)], to the [Binding](#) component. This IRI refers to an appropriate SOAP underlying protocol binding (see SOAP Protocol Binding Framework in [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)]), which is to be used for any of the SOAP interactions described by this binding.

5.5.3 XML Representation

```

<description>
  <binding name="xs:NCName" interface="xs:QName"? type="xs:anyURI"
    wssoap:protocol="xs:anyURI" >
    ...
  </binding>
</description>

```

The XML representation for specifying the SOAP protocol is a REQUIRED *attribute information item* with the following Infoset properties:

- A [local name] of `protocol`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/soap"`
- A type of `xs:anyURI`

5.5.4 Mapping from XML Representation to Component Properties

See [Table 5-2](#).

Table 5-2. Mapping from XML Representation to Binding component Extension Properties

Property	Value
{ soap underlying protocol }	The actual value of the <code>wssoap:protocol</code> <i>attribute information item</i> .

5.6 Binding Faults

5.6.1 Description

For every [Interface Fault](#) component contained in an [Interface](#) component, a mapping to a SOAP Fault MUST be described.[†] This binding extension specification allows the user to indicate the SOAP fault code and subcodes that are transmitted for a given [Interface Fault](#) component.

5.6.2 Relationship to WSDL Component Model

The SOAP Fault binding extension adds the following properties to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {soap fault code} REQUIRED. A union of `xs:QName` and `xs:token`, to the [Binding Fault](#) component, where:
 - when the value of the {[soap version](#)} is "1.2", the allowed QNames MUST be the ones defined by [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)], section 5.4.6[†];
 - the allowed token value is `"#any"`.

The value of this property identifies a possible SOAP fault for the operations in scope. If the value of this property is `"#any"`, no assertion is made about the possible value of the SOAP fault code.

- {soap fault subcodes} REQUIRED. A union of list of `xs:QName`, and `xs:token` where the allowed token value is `"#any"`, to the [Binding Fault](#) component. The value of this property identifies one or more subcodes for this SOAP fault. The list of subcodes is the nested sequence of subcodes. An empty list represents a fault code without subcodes.

5.6.3 XML Representation

```

<description>
  <binding >
    <fault ref="xs:QName"

```

```

    wsoap:code="union of xs:QName, xs:token"?
    wsoap:subcodes="union of (list of xs:QName), xs:token"? >
  <documentation />*
</fault>*
</binding>
</description>

```

The XML representation for binding a SOAP Fault are two *attribute information items* with the following Infoset properties:

- wsoap:code OPTIONAL *attribute information item*
 - A [local name] of code
 - A [namespace name] of "http://www.w3.org/ns/wsdl/soap"
 - A type of union of xs:QName and xs:token where the allowed token value is "#any"
- wsoap:subcodes OPTIONAL *attribute information item*
 - A [local name] of subcodes
 - A [namespace name] of "http://www.w3.org/ns/wsdl/soap"
 - A type of union of list of xs:QName, and xs:token where the allowed token value is "#any"

5.6.4 Mapping XML Representation to Component Properties

See [Table 5-3](#).

Table 5-3. Mapping from XML Representation to SOAP Fault component Properties

Property	Value
{ soap fault code }	The actual value of the <i>code attribute information item</i> , if present; otherwise "#any".
{ soap fault subcodes }	The actual value of the <i>subcodes attribute information item</i> , if present; otherwise "#any".

5.7 Binding Operations

5.7.1 Description

For every [Interface Operation](#) component contained in an [Interface](#) component, in addition to the binding rules (for SOAP 1.2, see [5.10.3 SOAP 1.2 Binding Rules](#)), there may be additional binding information to be specified. This binding extension specification allows the user to indicate the SOAP Message Exchange Pattern (MEP) and a value for the SOAP Action Feature on a per-operation basis.

5.7.2 Relationship to WSDL Component Model

The SOAP Operation binding extension specification adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {soap mep default} OPTIONAL. A xs:anyURI, which is an absolute IRI as defined by [[IETF RFC 3987](#)], to the [Binding](#) component.[‡] The value of this property identifies the default SOAP Message Exchange Pattern (MEP) for all the [Interface Operation](#) components of any [Interface](#) component to which this [Binding](#) is applied.
- {soap mep} OPTIONAL. A xs:anyURI, which is an absolute IRI as defined by [[IETF RFC 3987](#)], to the [Binding Operation](#) component.[‡] The value of this property identifies the SOAP Message Exchange Pattern (MEP) for this specific operation (see [5.10.3 SOAP 1.2 Binding Rules](#), paragraph "SOAP MEP Selection", for constraints on bindings).
- {soap action} OPTIONAL. A xs:anyURI, which is an absolute IRI as defined by [[IETF RFC 3987](#)], to the [Binding Operation](#) component.[‡] The value of this property identifies the value of the SOAP Action Feature for the initial message of the message exchange pattern of

the [Interface Operation](#) bound, as specified in the binding rules of bindings to specific versions of SOAP (see [5.10.3 SOAP 1.2 Binding Rules](#) for the SOAP 1.2 binding when the value of the `{soap:version}` property of the [Binding](#) component is "1.2").

5.7.3 XML Representation

```
<description>
  <binding wsoap:mepDefault="xs:anyURI"? >
    <operation ref="xs:QName"
      wsoap:mep="xs:anyURI"?
      wsoap:action="xs:anyURI"? >
    </operation>
  </binding>
</description>
```

The XML representation for binding a [Binding Operation](#) are two *attribute information items* with the following Infoset properties:

- wsoap:mep OPTIONAL *attribute information item*
 - A [local name] of mep
 - A [namespace name] of "http://www.w3.org/ns/wsdli/soap"
 - A type of xs:anyURI
- wsoap:action OPTIONAL *attribute information item*
 - A [local name] of action
 - A [namespace name] of "http://www.w3.org/ns/wsdli/soap"
 - A type of xs:anyURI

The following *attribute information item* for the binding *element information item* is defined:

- wsoap:mepDefault OPTIONAL *attribute information item*
 - A [local name] of mepDefault
 - A [namespace name] of " http://www.w3.org/ns/wsdli/soap "
 - A type of xs:anyURI

5.7.4 Mapping from XML Representation to Component Properties

See [Table 5-4](#).

Table 5-4. Mapping from XML Representation to SOAP Operation Component Properties

Property	Value
{soap:mepDefault}	The actual value of the wsoap:mepDefault <i>attribute information item</i> , if present.
{soap:mep}	The actual value of the wsoap:mep <i>attribute information item</i> , if present.
{soap:action}	The actual value of the wsoap:action <i>attribute information item</i> , if any.

5.8 Declaring SOAP Modules

5.8.1 Description

The SOAP messaging framework allows a Web service to engage one or more additional features (typically implemented as one or more SOAP header blocks), as defined by SOAP Modules (see [\[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)\]](#)). This binding extension specification allows description of which SOAP Modules are in use across an entire binding, on a per operation basis or on a per-message basis.

5.8.2 Relationship to WSDL Component Model

The [SOAP Module](#) component adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {soap modules} OPTIONAL. A set of [SOAP Module](#) components as defined in [5.8.3 SOAP Module component](#) to the [Binding](#) component
- Similarly, {soap modules} OPTIONAL, to the [Binding Operation](#) component
- Similarly, {soap modules} OPTIONAL, to the [Binding Message Reference](#) component
- Similarly, {soap modules} OPTIONAL, to the [Binding Fault](#) component
- Similarly, {soap modules} OPTIONAL, to the [Binding Fault Reference](#) component

The SOAP modules applicable for a particular operation of any service, consists of all the modules specified in the input or output [Binding Message Reference](#) components, the infault or outfault [Binding Fault Reference](#) components, those specified within the [Binding Fault](#) components, those specified within the [Binding Operation](#) components and those specified within the [Binding](#) component. If any module is declared in multiple components, then the requiredness of that module is defined by the closest declaration, where closeness is defined by whether it is specified directly at the [Binding Message Reference](#) component or [Binding Fault Reference](#) component level, the [Binding Fault](#) level or the [Binding Operation](#) component level or the [Binding](#) component level, respectively.

5.8.3 SOAP Module component

The [SOAP Module](#) component identifies a SOAP module that is in use.

The properties of the SOAP Module component are as follows:

- {ref} REQUIRED. A *xs:anyURI*, which is an absolute IRI as defined by [[IETF RFC 3987](#)].[†] The value of this property uniquely identifies the SOAP module that is in use (as per the SOAP 1.2 [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)] processing model).
- {required} REQUIRED. A *xs:boolean* indicating if the SOAP module is required.
- {parent} REQUIRED. The [Binding](#), [Binding Operation](#), [Binding Message Reference](#), [Binding Fault](#) or [Binding Fault Reference](#) components that contains this component in its {[soap modules](#)} property.

5.8.4 XML Representation

```

<description>
  <binding >
    <wssoap:module ref="xs:anyURI"
      required="xs:boolean"? >
      <documentation ... />*
    </wssoap:module>
    <fault>
      <wssoap:module ... />*
    </fault>
    <operation>
      <wssoap:module ... />*
      <input>
        <wssoap:module ... />*
      </input>
      <output>
        <wssoap:module ... />*
      </output>
      <infault>
        <wssoap:module ... />*
      </infault>
      <outfault>
        <wssoap:module ... />*
      </outfault>
    </operation>
  </binding>
</description>

```

The XML representation for a [SOAP Module](#) component is an *element information item* with the following Infoset properties:

- A [local name] of `module`
- A [namespace name] of "http://www.w3.org/ns/wsd/soap"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED `ref` *attribute information item* with the following Infoset properties:
 - A [local name] of `ref`
 - A [namespace name] which has no value
 - A type of `xs:anyURI`
 - An OPTIONAL `required` *attribute information item* with the following Infoset properties:
 - A [local name] of `required`
 - A [namespace name] which has no value
 - A type of `xs:boolean`
 - Zero or more namespace qualified *attribute information items*. The [namespace name] of such *attribute information items* MUST NOT be "http://www.w3.org/ns/wsd/" and MUST NOT be "http://www.w3.org/ns/wsd/soap".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* as defined in [[WSDL 2.0 Core Language](#)].
 2. Zero or more namespace-qualified *element information items* amongst its [children]. The [namespace name] of such *element information items* MUST NOT be "http://www.w3.org/ns/wsd/" and MUST NOT be "http://www.w3.org/ns/wsd/soap".

5.8.5 Mapping from XML Representation to Component Properties

See [Table 5-5](#).

Table 5-5. Mapping from XML Representation to SOAP Module component-related Properties

Property	Value
{soap modules}	The set of SOAP Module components corresponding to all the <code>module</code> <i>element information item</i> in the [children] of the binding, operation, fault, input, output, infault, outfault <i>element information items</i> , if any.
{ref}	The actual value of the <code>ref</code> <i>attribute information item</i> .
{required}	The actual value of the <code>required</code> <i>attribute information item</i> , if present; otherwise "false".
{parent}	The Binding , Binding Operation , Binding Message Reference , Binding Fault or Binding Fault Reference component corresponding to the binding, operation, fault, input, output, infault or outfault <i>element information item</i> in [parent].

5.8.6 IRI Identification Of A SOAP Module component

WSDL Version 2.0 Part 1: Core Language [[WSDL 2.0 Core Language](#)] defines a [fragment identifier syntax](#) for identifying components of a WSDL 2.0 document.

A [SOAP Module](#) component can be identified using the [wsdl.extension](#) XPointer Framework scheme:

```
wsdl.extension(http://www.w3.org/ns/wsd/soap, wsoap.module(parent/ref))
```

1. *parent* is the pointer part of the [{parent}](#) component, as specified in [appendix A.2, Fragment Identifiers](#) in [[WSDL 2.0 Core Language](#)]. parts.
2. *ref* is the value of the [{ref}](#) property of the component.

5.9 Declaring SOAP Header Blocks

5.9.1 Description

SOAP allows the use of header blocks in the header part of the message. This binding extension allows users to declare the SOAP header blocks in use on a per-message and on a per-fault basis.

5.9.2 Relationship to WSDL Component Model

The SOAP Header Blocks binding extension specification adds the following property to the WSDL component model (as defined in [\[WSDL 2.0 Core Language\]](#)):

- {soap headers} OPTIONAL. A set of [SOAP Header Block](#) components as defined in [5.9.3 SOAP Header Block component](#), to the [Binding Message Reference](#) component.
- Similarly, {soap headers} OPTIONAL, to the [Binding Fault](#) component.

5.9.3 SOAP Header Block component

A [SOAP Header Block](#) component describes an abstract piece of header data (SOAP header block) that is associated with the exchange of messages between the communicating parties. The presence of a [SOAP Header Block](#) component in a WSDL description indicates that the service supports headers, and MAY require a client interacting with the service to use the described header block. Zero or one such header block may be used.

The properties of the SOAP Header Block component are as follows:

- {element declaration} REQUIRED. An XML element declaration in the {[element declarations](#)} property of the [Description](#) component. This XML element declaration uniquely represents a specific SOAP header block.
- {mustUnderstand} REQUIRED. A *xs:boolean*. When its value is "true", the SOAP header block MUST be decorated with a SOAP *mustUnderstand attribute information item* with a value of "true"; if so, the XML element declaration referenced by the {[element declaration](#)} property MUST allow this SOAP *mustUnderstand attribute information item*.[†] Otherwise, no additional constraint is placed on the presence and value of a SOAP *mustUnderstand attribute information item*.
- {required} REQUIRED. A *xs:boolean* indicating if the SOAP header block is required. If the value is "true", then the SOAP header block MUST be included in the message.[†] If it is "false", then the SOAP header block MAY be included.
- {parent} REQUIRED. The [Binding Fault](#) or [Binding Message Reference](#) component that contains this component in its {[soap headers](#)} property.

5.9.4 XML Representation

```

<description>
  <binding name="xs:NCName" type="http://www.w3.org/ns/wsdL/soap" >
    <fault ref="xs:QName" >
      <wsoap:header element="xs:QName" mustUnderstand="xs:boolean"?
        required="xs:boolean"? >
        <documentation /*>
      </wsoap:header>*
    </fault>*
    <operation ref="xs:QName" >
      <input messageLabel="xs:NCName"?>
        <wsoap:header ... /*>
      </input>*
      <output messageLabel="xs:NCName"?>
        <wsoap:header ... /*>
      </output>*
    </operation>*
  </binding>
</description>

```

The XML representation for a [SOAP Header Block](#) component is an *element information item* with the following Infoset properties:

- A [local name] of header
- A [namespace name] of "http://www.w3.org/ns/wsdl/soap"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *element attribute information item* with the following Infoset properties:
 - A [local name] of element
 - A [namespace name] which has no value
 - A type of *xs:QName*
 - An OPTIONAL *mustUnderstand attribute information item* with the following Infoset properties:
 - A [local name] of mustUnderstand
 - A [namespace name] which has no value
 - A type of *xs:boolean*
 - An OPTIONAL *required attribute information item* with the following Infoset properties:
 - A [local name] of required
 - A [namespace name] which has no value
 - A type of *xs:boolean*
 - Zero or more namespace qualified *attribute information items*. The [namespace name] of such *attribute information items* MUST NOT be "http://www.w3.org/ns/wsdl" and MUST NOT be "http://www.w3.org/ns/wsdl/soap".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* as defined in [\[WSDL 2.0 Core Language\]](#).
 2. Zero or more namespace-qualified *element information items* amongst its [children]. The [namespace name] of such *element information items* MUST NOT be "http://www.w3.org/ns/wsdl" and MUST NOT be "http://www.w3.org/ns/wsdl/soap".

5.9.5 Mapping XML Representation to Component Properties

See [Table 5-6](#).

Table 5-6. Mapping from XML Representation to SOAP Header Block component-related Properties

Property	Value
{soap headers}	The set of SOAP Header Block components corresponding to all the header <i>element information item</i> in the [children] of the fault, input OR output <i>element information item</i> , if any.
{element declaration}	The element declaration from the {element declarations} resolved to by the value of the <i>element attribute information item</i> . The value of the <i>element attribute information item</i> MUST resolve to a global element declaration from the {element declarations} property of the Description component. [†]
{mustUnderstand}	The actual value of the <i>mustUnderstand attribute information item</i> , if present; otherwise "false".
{required}	The actual value of the <i>required attribute information item</i> , if present; otherwise "false".
{parent}	The Binding Fault or Binding Message Reference component corresponding to the fault, input OR output <i>element information item</i> in [parent].

5.9.6 IRI Identification Of A SOAP Header Block component

WSDL Version 2.0 Part 1: Core Language [[WSDL 2.0 Core Language](#)] defines a [fragment identifier syntax](#) for identifying components of a WSDL 2.0 document.

A [SOAP Header Block](#) component can be identified using the [wsdl:extension](#) XPointer Framework scheme:

```
wsdl:extension(http://www.w3.org/ns/wsdl/soap, wsoap.header(parent/element declaration))
```

1. *parent* is the "wsdl.*" pointer part of the {[parent](#)} component, as specified in [appendix A.2, Fragment Identifiers](#) in [[WSDL 2.0 Core Language](#)], i.e. without the xmlns() pointer parts.
2. *element declaration* is the value of the {[name](#)} of the Element Declaration component that is referred to by the {[element declaration](#)} property of the SOAP Header Block component.

5.10 WSDL SOAP 1.2 Binding

This section describes the SOAP 1.2 binding for WSDL 2.0. This binding does NOT natively support the full range of capabilities from SOAP 1.2. Certain capabilities not widely used, or viewed as problematic in practice, are not available -in many cases because supporting them was considered as adding considerable complexity to the language. Here are examples of such unsupported capabilities:

- multiple children of the SOAP Body;
- multiple SOAP Fault Detail entries;
- non-qualified elements as children of a SOAP Fault Detail.

5.10.1 Identifying a WSDL SOAP 1.2 Binding

A WSDL SOAP Binding is identified as a SOAP 1.2 binding by assigning the value "1.2" to the {[soap version](#)} property of the [Binding](#) component.

5.10.2 Description

The WSDL SOAP 1.2 binding extension defined in this section is an extension of the SOAP binding defined in section [5. WSDL SOAP Binding Extension](#) to enable Web service applications to use SOAP 1.2 [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)].

The WSDL SOAP 1.2 binding extension supports the SOAP 1.2 HTTP binding defined by the [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)] specification. This is indicated by assigning the URI "http://www.w3.org/2003/05/soap/bindings/HTTP/" (as defined by [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)]) to the {[soap underlying.protocol](#)} property. Other values MAY be used for this property in conjunction with the SOAP 1.2 binding extension defined by this specification provided that the semantics of such protocols are consistent with this binding extension.

Default rules in section [5.10.3 SOAP 1.2 Binding Rules](#) define the relationship between SOAP message exchange patterns defined in [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)] and WSDL message exchange patterns defined in section [2. Predefined Message Exchange Patterns](#).

5.10.3 SOAP 1.2 Binding Rules

These binding rules are applicable to SOAP 1.2 bindings.

- *SOAP Action Feature*. The value of the SOAP Action Feature for the initial message of the message exchange pattern of the [Interface Operation](#) bound is specified by the {[soap action](#)} property of this [Binding Operation](#) component. If the [Binding Operation](#) component does NOT have a {[soap action](#)} property defined, then the SOAP Action Feature (see [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)]) has NO value. Otherwise, its value is the value of the SOAP Action Feature for the initial message of the message exchange pattern. The {[soap action](#)} property has NO effect when binding to the SOAP-Response MEP.

- **SOAP MEP Selection.** For a given [Interface Operation](#) component, if there is a [Binding Operation](#) component whose `{interface operation}` property matches the component in question and its `{soap mep}` property has a value, then the SOAP MEP is the value of the `{soap mep}` property. Otherwise, the SOAP MEP is the value of the [Binding](#) component's `{soap mep default}`, if any. Otherwise, the [Interface Operation](#) component's `{message exchange pattern}` property MUST have the value "http://www.w3.org/ns/wsdl/in-out", and the SOAP MEP is the URI "http://www.w3.org/2003/05/soap/mep/request-response/" identifying the SOAP Request-Response Message Exchange Pattern as defined in [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)].[‡]
- **SOAP Detail Element.** If any, the value of the SOAP "Detail" element MUST be the *element information item* identified by the `{element declaration}` property of the [Interface Fault](#) component.[‡]
- **HTTP Method Selection.** This default binding rule is applicable when the value of the `{soap underlying protocol}` property of the [Binding](#) component is "http://www.w3.org/2003/05/soap/bindings/HTTP/". If the SOAP MEP selected as specified above has the value "http://www.w3.org/2003/05/soap/mep/request-response/" then the HTTP method used is "POST". If the SOAP MEP selected has the value "http://www.w3.org/2003/05/soap/mep/soap-response/" then the HTTP method used is "GET".[‡]

5.10.4 Binding WSDL 2.0 MEPs to SOAP 1.2 MEPs

This section describes the relationship between WSDL components and SOAP 1.2 MEP properties as described in [[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)].

5.10.4.1 WSDL In-Out to SOAP Request-Response

This section describes the mapping from the WSDL "http://www.w3.org/ns/wsdl/in-out" Message Exchange Pattern (MEP) to the SOAP "http://www.w3.org/2003/05/soap/mep/request-response/" MEP (as would be the case for a usual SOAP-over-HTTP In-Out operation). Extensions (such as [[WSA 1.0 Core](#)]) MAY alter these mappings.

5.10.4.1.1 THE CLIENT

As the client, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RequestingSOAPNode".

The SOAP "http://www.w3.org/2003/05/soap/mep/ImmediateDestination" property takes the value of the HTTP Request IRI, as defined in [6.4.6 HTTP Request IRI](#), and modified as described in section [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#).

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/OutboundMessage" property.

The WSDL "Out" message maps to the SOAP

"http://www.w3.org/2003/05/soap/mep/InboundMessage" property.

5.10.4.1.2 THE SERVICE

As the service, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RespondingSOAPNode".

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/InboundMessage" property.

The WSDL "Out" message maps to the SOAP
"http://www.w3.org/2003/05/soap/mep/OutboundMessage" property.

5.10.4.2 WSDL In-Out to SOAP SOAP-Response

This section describes the mapping from the WSDL "http://www.w3.org/ns/wsdl/in-out" MEP to the "http://www.w3.org/2003/05/soap/mep/soap-response/" SOAP MEP. Extensions (such as [\[WSA 1.0 Core\]](#)) MAY alter these mappings.

5.10.4.2.1 THE CLIENT

As the client, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RequestingSOAPNode".

The SOAP "http://www.w3.org/2003/05/soap/mep/ImmediateDestination" property takes the value of the HTTP Request IRI, as defined in [6.4.6 HTTP Request IRI](#), and modified as described in section [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#).

The value of the {[message content model](#)} property for the [Interface Message Reference](#) components of the {[interface message references](#)} property MUST be either "#element" or "#none". When the value is:

- "#element", the WSDL "In" message is mapped to the destination URI, as per the rules in section [6.8.2 Serialization as application/x-www-form-urlencoded](#).
- "#none", the WSDL "In" message is empty.

The SOAP "http://www.w3.org/2003/05/soap/mep/OutboundMessage" property has no value.

The WSDL "Out" message maps to the SOAP

"http://www.w3.org/2003/05/soap/mep/InboundMessage" property.

5.10.4.2.2 THE SERVICE

As the service, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RespondingSOAPNode".

The WSDL "In" message is constructed from the destination URI as per the rules in section [6.8.2 Serialization as application/x-www-form-urlencoded](#), WHEN the value of the {[message content model](#)} property for the [Interface Message Reference](#) components of the {[interface message references](#)} property is "#element".

The WSDL "Out" message maps to the SOAP

"http://www.w3.org/2003/05/soap/mep/OutboundMessage" property.

5.10.4.3 WSDL In-Only to SOAP Request-Response

This section describes the mapping from the WSDL "http://www.w3.org/ns/wsdl/in-only" MEP to the SOAP "http://www.w3.org/2003/05/soap/mep/request-response/" MEP. Extensions (such as [\[WSA 1.0 Core\]](#)) MAY alter these mappings.

5.10.4.3.1 THE CLIENT

As the client, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value

"RequestingSOAPNode".

The SOAP "http://www.w3.org/2003/05/soap/mep/ImmediateDestination" property takes the value of the HTTP Request IRI, as defined in [6.4.6 HTTP Request IRI](#), and modified as described in section [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#).

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/OutboundMessage" property.

The SOAP "http://www.w3.org/2003/05/soap/mep/InboundMessage" property has no value.

5.10.4.3.2 THE SERVICE

As the service, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RespondingSOAPNode".

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/InboundMessage" property.

The SOAP "http://www.w3.org/2003/05/soap/mep/OutboundMessage" property has no value.

5.10.4.4 WSDL Robust-In-Only to SOAP Request-Response

This section describes the mapping from the WSDL "http://www.w3.org/ns/wsd/robust-in-only" MEP to the SOAP "http://www.w3.org/2003/05/soap/mep/request-response/" MEP. Extensions (such as [\[WSA 1.0 Core\]](#)) MAY alter these mappings.

5.10.4.4.1 THE CLIENT

As the client, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RequestingSOAPNode".

The SOAP "http://www.w3.org/2003/05/soap/mep/ImmediateDestination" property takes the value of the HTTP Request IRI, as defined in [6.4.6 HTTP Request IRI](#), and modified as described in section [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#).

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/OutboundMessage" property.

The SOAP "http://www.w3.org/2003/05/soap/mep/InboundMessage" can contain a SOAP fault.

5.10.4.4.2 THE SERVICE

As the service, the property

"http://www.w3.org/2003/05/soap/bindingFramework/ExchangeContext/Role" takes the value "RespondingSOAPNode".

The WSDL "In" message is mapped to the SOAP

"http://www.w3.org/2003/05/soap/mep/InboundMessage" property.

The SOAP "http://www.w3.org/2003/05/soap/mep/OutboundMessage" can contain a SOAP fault.

5.11 Conformance

An *element information item* whose namespace name is "http://www.w3.org/ns/wsd/" and whose local part is *description* conforms to this binding extension specification if the *element*

information items and *attribute information items* whose namespace is `http://www.w3.org/ns/wsdl/soap` conform to the XML Schema for that element or attribute as defined by this specification and additionally adheres to all the constraints contained in this specification.

6. WSDL HTTP Binding Extension

The HTTP binding extension described in this section is an extension for [[WSDL 2.0 Core Language](#)] to enable Web services applications to use HTTP 1.1 [[IETF RFC 2616](#)] (as well as other versions of HTTP) and HTTPS [[IETF RFC 2818](#)]. This binding extension extends WSDL 2.0 by adding properties to the component model defined in [[WSDL 2.0 Core Language](#)]. In addition an XML Infoset representation for these additional properties is provided, along with a mapping from that representation to the various component properties.

As allowed in [[WSDL 2.0 Core Language](#)], a [Binding](#) component can exist without indicating a specific [Interface](#) component that it applies to and, in this case, no [Binding Operation](#) or [Binding Fault](#) components can be present in the [Binding](#) component.

The HTTP binding extension is designed with the objective of minimizing what needs to be explicitly declared for common cases. This is achieved by defining a set of default rules that affect all [Interface Operation](#) components of an [Interface](#) component to which the HTTP binding extension is applied, unless specifically overridden by a [Binding Operation](#) component. Thus, if a given [Interface Operation](#) component is not referred to specifically by a [Binding Operation](#) component, then all the default rules apply to that [Interface Operation](#) component. As a result, in accordance with the requirements of [[WSDL 2.0 Core Language](#)], all operations of an [Interface](#) component will be bound by this binding extension.

Note: As in other parts of this specification, one could have done away with "default" properties at the component model level, and have set the value for the corresponding non-default properties in the XML mapping section. However, default properties are required for interface-less binding. Indeed, an interface-less binding has no means to set the non-default version of the property at the operation-level, since there is precisely no operation (there is not even an interface). Hence the mapping needs to be done elsewhere.

[Definition: The internal tree representation of an input, output or fault message is called an **instance data**, and is constrained by the schema definition associated with the message: the XML element referenced in the {[element declaration](#)} property of the [Interface Message Reference](#) component for input and output messages (unless the {[message content model](#)} is "#any"), and in the {[element declaration](#)} property of an [Interface Fault](#) component for faults.]

6.1 Identifying the use of the HTTP Binding

A [Binding](#) component (defined in [[WSDL 2.0 Core Language](#)]) is identified as an HTTP binding by assigning the value `"http://www.w3.org/ns/wsdl/http"` to the {[type](#)} property of the [Binding](#) component.

6.2 HTTP Syntax Summary (Non-Normative)

```
<description>
  <binding name="xs:NCName" interface="xs:QName"?
    type="http://www.w3.org/ns/wsdl/http"
    whttp:methodDefault="xs:string"?
    whttp:queryParameterSeparatorDefault="xs:string"?
    whttp:cookies="xs:boolean"?
    whttp:contentEncodingDefault="xs:string"? >
  <documentation />?

  <fault ref="xs:QName"
    whttp:code="union of xs:int, xs:token"?
    whttp:contentEncoding="xs:string"? >
  <documentation />*
  <whttp:header name="xs:string" type="xs:QName"
    required="xs:boolean"? >
  <documentation />*
  </whttp:header>*
</fault>*
```

```

<operation ref="xs:QName"
  whttp:location="xs:anyURI"?
  whttp:method="xs:string"?
  whttp:inputSerialization="xs:string"?
  whttp:outputSerialization="xs:string"?
  whttp:faultSerialization="xs:string"?
  whttp:queryParameterSeparator="xs:string"?
  whttp:contentEncodingDefault="xs:string"?
  whttp:ignoreUncited="xs:boolean"? >
  <documentation />*

  <input messageLabel="xs:NCName"?
    whttp:contentEncoding="xs:string"? >
    <documentation />*
    <whttp:header ... />*
  </input>*

  <output messageLabel="xs:NCName"?
    whttp:contentEncoding="xs:string"? >
    <documentation />*
    <whttp:header ... />*
  </output>*

  <infault ref="xs:QName"
    messageLabel="xs:NCName"? >
    <documentation />*
  </infault>*

  <outfault ref="xs:QName"
    messageLabel="xs:NCName"? >
    <documentation />*
  </outfault>*

</operation>*

</binding>

<service>
  <endpoint name="xs:NCName" binding="xs:QName" address="xs:anyURI"?
    whttp:authenticationScheme="xs:token"?
    whttp:authenticationRealm="xs:string"? >
    <documentation />*
  </endpoint>
</service>
</description>

```

6.3 Supported Extensions

An implementation of the HTTP binding extension **MUST** support the following extensions:

- "http://www.w3.org/ns/wsdl-extensions/safe" (see [3.1 Operation safety](#))

6.4 HTTP Binding Rules

6.4.1 HTTP Method Selection

When formulating the HTTP message to be transmitted, the HTTP request method used **MUST** be selected using one of the following:[‡]

- For a given [Interface Operation](#) component, if there is a [Binding Operation](#) component whose {[interface operation](#)} property matches the component in question and its {[http method](#)} property has a value, then the value of the {[http method](#)} property.
- Otherwise, the value of the [Binding](#) component's {[http method default](#)}, if any.
- Otherwise, if a {[safe](#)} property as defined in [3.1 Operation safety](#) is present on the bound [Interface Operation](#) component and has a value of "true", the value "GET".
- Otherwise, the value "POST".

6.4.2 HTTP Content Encoding Selection

When formulating the HTTP message to be transmitted, content encoding for a given [Binding Message Reference](#) component is determined as follows:[‡]

- If the [{http content encoding}](#) property has a non-empty value, a Content-Encoding header-field MUST be inserted with the value of this property.
- Otherwise, if the value of the parent [Binding Operation](#) component's [{http content encoding default}](#) property has a non-empty value, a Content-Encoding header-field MUST be inserted with the value of this property.
- Otherwise, if the value of the grandparent [Binding](#) component's [{http content encoding default}](#) property has a non-empty value, a Content-Encoding header-field MUST be inserted with the value of this property.

When formulating the HTTP fault message to be transmitted, content encoding for a given [Binding Fault](#) component is determined as follows:[‡]

- If the [{http content encoding}](#) property has a non-empty value, then a Content-Encoding header-field MUST be inserted with the value of this property.
- If the [{http content encoding default}](#) property has a non-empty value, then a Content-Encoding header-field MUST be inserted with the value of this property.

The body of the response message is encoded using the specified content encoding.

6.4.3 Payload Construction And Serialization Format

When formulating the HTTP message to be transmitted, the contents of the payload (i.e. the contents of the HTTP message body) MUST be what is defined by the corresponding [Interface Message Reference](#) or [Interface Fault](#) components, serialized as specified by the [serialization format](#) used.[‡]

[Definition: The **serialization format** is a media type token ("type/subtype"). It identifies rules to serialize the payload in an HTTP message. Its value is defined by the following rules. The HTTP request serialization format MUST be in the media type range specified by the [{http input serialization}](#) property. The HTTP response serialization format MUST be in the media type range specified by the [{http output serialization}](#) property. The HTTP serialization format of a fault MUST be in the media type range specified by the [{http fault serialization}](#) property. The concept of media type range is defined in Section 14.1 of [\[IETF RFC 2616\]](#). The serialization format MAY have **associated media type parameters** (specified with the parameter production of media-range in Section 14.1 of [\[IETF RFC 2616\]](#).]

Section [6.8 Serialization Format of Instance Data](#) defines serialization formats supported by this binding extension along with their constraints.

- [Interface Message Reference](#) component:
 - If the value of the [{message content model}](#) property of the [Interface Message Reference](#) bound is "#any" or "#element", the serialization of the instance data is specified as defined in section [6.4.3.1 Serialization rules for XML messages](#).
 - If the value is "#none", then the payload MUST be empty and the value of the corresponding serialization property ([{http input serialization}](#) or [{http output serialization}](#)) is ignored.[‡]
 - If the value is "#other", then the [serialization format](#) and its associated media type parameters, if any, specifies the value of the HTTP Content-Type entity-header field as defined in section 14.17 of [\[IETF RFC 2616\]](#). The serialization of the payload is undefined.
- [Interface Fault](#) component: the serialization of the instance data is specified as defined in section [6.4.3.1 Serialization rules for XML messages](#).

If the [Interface Message Reference](#) component or the [Interface Fault](#) component is declared using a non-XML type system (as considered in the Types section of [\[WSDL 2.0 Core](#)

[Language](#)]), then additional binding rules **MUST** be defined in an extension specification to indicate how to map those components into the HTTP envelope.[‡]

6.4.3.1 Serialization rules for XML messages

The serialization rules for messages whose [{message content model}](#) is either "#element" or "#any", AND the serialization rules for fault messages, are as follows:[‡]

- If the [serialization format](#) is "application/x-www-form-urlencoded", then the serialization of the [instance data](#) is defined by section [6.8.2 Serialization as application/x-www-form-urlencoded](#).
- If the [serialization format](#) is "multipart/form-data", then the serialization of the [instance data](#) is defined by section [6.8.4 Serialization as multipart/form-data](#).
- If the [serialization format](#) is "application/xml", then the serialization of the [instance data](#) is defined by section [6.8.3 Serialization as application/xml](#).
- Otherwise, then the serialization of the [instance data](#) is defined by section [6.8.3 Serialization as application/xml](#) with the following additional rule: the value of the HTTP Content-Type entity-header field is the value of the [serialization format](#) and its associated media type parameters, if any.

6.4.4 Default input and output serialization format

Section [Table 6-1](#) defines the default values for the GET, POST, PUT and DELETE values of the HTTP method as selected in section [6.4.1 HTTP Method Selection](#).

Table 6-1. Default values for GET, POST, PUT and DELETE

HTTP Method	Default Input Serialization	Default Output Serialization
Selected in 6.4.1 HTTP Method Selection	{http input serialization}	{http output serialization}
GET	application/x-www-form-urlencoded	application/xml
POST	application/xml	application/xml
PUT	application/xml	application/xml
DELETE	application/x-www-form-urlencoded	application/xml

Note:

The application/x-www-form-urlencoded serialization format places constraints on the XML Schema definition of the [{element declaration}](#) property of the [Interface Message Reference](#) components of the [Interface Operation](#) component bound (see [6.8.2 Serialization as application/x-www-form-urlencoded](#)).

The default value for the [{http input serialization}](#) and [{http output serialization}](#) properties for any other HTTP method selected is application/xml.

Mechanisms other than setting the serialization properties **MAY** modify the serialization format of the [instance data](#) corresponding to the message. An example of such modification is the WSDL SOAP Binding HTTP IRI Serialization rules specified in [5.3 SOAP Binding Rules](#). This binding extension specifies that the [SOAP-Response Message Exchange Pattern](#) ([[SOAP 1.2 Part 2: Adjuncts \(Second Edition\)](#)], Section 6.3) supports input message serialization only as application/x-www-form-urlencoded. Other examples are other message exchange patterns or binding extensions.

6.4.5 HTTP Header Construction

If the [{http headers}](#) property as defined in section [6.6 Declaring HTTP Headers](#) exists and is not empty in a [Binding Message Reference](#) or [Binding Fault](#) component, HTTP headers

conforming to each [HTTP Header](#) component contained in this `{http_headers}` property MAY be serialized as follows:[‡]

- The HTTP header field name used is the value of the `{name}` property of the [HTTP Header](#) component. The HTTP binding MUST NOT set an HTTP header field corresponding to the value of the `{name}` property already set by another mechanism, such as the HTTP stack or another feature.[‡]
- The HTTP header field value, whose XML Schema type is declared by the `{type_definition}` property of the [HTTP Header](#) component, is serialized following the rules of the `field-value` production of section 4.2 of [\[IETF RFC 2616\]](#).

If the value of an [HTTP Header](#) component's `{required}` property is "true", the inclusion of this HTTP header field is REQUIRED[‡], otherwise it is OPTIONAL.

6.4.6 HTTP Request IRI

When formulating the HTTP Request, the HTTP Request IRI is an absolute IRI reference and is the value of the `{http_location}` property of the [Binding Operation](#) component, resolved using the value of the `{address}` property of the [Endpoint](#) component (see section 5 of [\[IETF RFC 3986\]](#)).[‡] If the `{http_location}` property is not set, the HTTP Request IRI is the value of the `{address}` property of the [Endpoint](#) component. Input serializations may define additional processing rules to be applied to the value of `{http_location}` before applying the process of reference resolution, i.e. before combining it with the `{address}` property of the endpoint element to form the HTTP Request IRI. For example, the three serialization formats defined in section [6.8 Serialization Format of Instance Data](#) define a syntax to use the `{http_location}` as a template using elements of the instance data.

If the resulting IRI uses the `https` scheme, then HTTP over TLS [\[IETF RFC 2818\]](#) is used to send the HTTP request.

The HTTP Request IRI identifies the resource upon which to apply the request and is transmitted using the Request-URI, and optionally the Host header field, as defined in [\[IETF RFC 2616\]](#).

6.5 Binding Operations

6.5.1 Description

This binding extension specification provides a binding to HTTP of [Interface Operation](#) components whose `{message_exchange_pattern}` property has a value amongst:

- "http://www.w3.org/ns/wsd/in-only"
- "http://www.w3.org/ns/wsd/robust-in-only"
- "http://www.w3.org/ns/wsd/in-out"

This HTTP binding extension MAY be used with other message exchange patterns, such as outbound message exchange patterns, provided that additional semantics are defined, for example through an extension.

Each of the three supported message exchange patterns above involves one or two messages or faults being exchanged. The first one is transmitted using an HTTP request, and the second one is transmitted using the corresponding HTTP response.[‡] In cases where only one single message is being sent, the message body of the HTTP response MUST be empty.[‡]

For successful responses, the HTTP response code MUST be:

- 202 when the MEP is "http://www.w3.org/ns/wsd/in-only"[‡]
- 204 when the MEP is "http://www.w3.org/ns/wsd/robust-in-only"[‡]

For every [Binding Operation](#) component corresponding to such [Interface Operation](#) components, this binding extension specification allows the user to indicate the HTTP method to

use, the input, output and fault serialization, and the location of the bound operation.

6.5.2 Relationship to WSDL Component Model

The HTTP binding extension adds the following properties to the WSDL component model (as defined in [\[WSDL 2.0 Core Language\]](#)):

- {http location} OPTIONAL. An *xs:anyURI*, to the [Binding Operation](#) component. It MUST contain an IRI reference and MUST NOT include a fragment identifier component.[‡]
- {http method default} OPTIONAL. A *xs:string*, to the [Binding](#) component, indicating the default value for the HTTP Request Method for all the [Interface Operation](#) components of any [Interface](#) component to which this [Binding](#) is applied.
- {http method} OPTIONAL. A *xs:string*, to the [Binding Operation](#) component, indicating the value for the HTTP Request Method for this specific [Binding Operation](#).
- {http input serialization} REQUIRED. A *xs:string*, to the [Binding Operation](#) component, indicating allowed serialization rules of the HTTP Request message for this specific operation, as described in section [6.5.3 Specification of serialization rules allowed](#).
- {http output serialization} REQUIRED. A *xs:string*, to the [Binding Operation](#) component, indicating allowed serialization rules of the HTTP Response message for this specific operation, as described in section [6.5.3 Specification of serialization rules allowed](#).
- {http fault serialization} REQUIRED. A *xs:string*, to the [Binding Operation](#) component, indicating allowed serialization rules of the HTTP Response message for this specific operation in case a fault is returned, as described in section [6.5.3 Specification of serialization rules allowed](#).
- {http query parameter separator default} REQUIRED. A *xs:string*, to the [Binding](#) component, indicating the default query parameter separator character for all the [Interface Operation](#) components of any [Interface](#) component to which this [Binding](#) is applied to.
- {http query parameter separator} OPTIONAL. A *xs:string*, to the [Binding Operation](#) component, indicating the query parameter separator character for this [Binding Operation](#).

6.5.3 Specification of serialization rules allowed

The value of the {[http input serialization](#)}, {[http output serialization](#)} and {[http fault serialization](#)} properties is similar to the value allowed for the Accept HTTP header defined by the HTTP 1.1 specification, Section 14.1 (see [\[IETF RFC 2616\]](#)) and MUST follow the production rules defined in that section except for the following:[‡]

1. The prefix "Accept:" MUST NOT be used.
2. The rule qdtext is changed from:

qdtext = <any TEXT except">">

to:

qdtext = <any CHAR except">">

This change is made to disallow non-US-ASCII OCTETs.

These properties indicate the range of media types and associated parameters with which an instance MAY be serialized. The value of the [serialization format](#) used for a message is a media type which MUST be covered by this range.[‡] Wild cards (for example, "application/*") SHOULD NOT be used in this *attribute information item* since they may lead to interoperability problems.[‡] The use of {[http input serialization](#)}, {[http output serialization](#)} and {[http fault serialization](#)} is specified in section [6.4.3 Payload Construction And Serialization Format](#).

6.5.4 XML Representation

```
<description>
  <binding http:methodDefault="xs:string"?
    http:queryParameterSeparatorDefault="xs:string"? >
    <operation ref="xs:QName"
```

```

        http:location="xs:anyURI"?
        http:method="xs:string"?
        http:inputSerialization="xs:string"?
        http:outputSerialization="xs:string"?
        http:faultSerialization="xs:string"?
        http:queryParameterSeparator="xs:string"? >
    </operation>
</binding>
</description>

```

The XML representation for binding an Operation are six *attribute information items* with the following Infoset properties:

- An OPTIONAL `location` *attribute information item* with the following Infoset properties:
 - A [local name] of `location`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:anyURI`
- An OPTIONAL `method` *attribute information item* with the following Infoset properties:
 - A [local name] of `method`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string`
- An OPTIONAL `inputSerialization` *attribute information item* with the following Infoset properties:
 - A [local name] of `inputSerialization`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string`
- An OPTIONAL `outputSerialization` *attribute information item* with the following Infoset properties:
 - A [local name] of `outputSerialization`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string`
- An OPTIONAL `faultSerialization` *attribute information item* with the following Infoset properties:
 - A [local name] of `faultSerialization`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string`
- An OPTIONAL `queryParameterSeparator` *attribute information item* with the following Infoset properties:
 - A [local name] of `queryParameterSeparator`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string` whose pattern facet is `"[&a-zA-Z0-9\-\.\_~!$'\"():@/\?*\+,]{1,1}"`, "&" and ";" being the most frequently used characters in practice.

The following *attribute information items* for the binding *element information item* are defined:

- An OPTIONAL `methodDefault` *attribute information item* with the following Infoset properties:
 - A [local name] of `methodDefault`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`
 - A type of `xs:string`
- An OPTIONAL `queryParameterSeparatorDefault` *attribute information item* with the following Infoset properties:
 - A [local name] of `queryParameterSeparatorDefault`
 - A [namespace name] of `"http://www.w3.org/ns/wsdll/http"`

- A type of `xs:string` whose length facet value is "1". The allowed characters are the same as for the `{http:query:parameter:separator}` property above.

6.5.5 Mapping from XML Representation to Component Properties

See [Table 6-2](#).

Table 6-2. Mapping from XML Representation to Binding Operation component Extension Properties

Property	Value
<code>{http:location}</code>	The actual value of the <code>whhttp:location</code> <i>attribute information item</i> , if present.
<code>{http:method:default}</code>	The actual value of the <code>whhttp:methodDefault</code> <i>attribute information item</i> , if present.
<code>{http:method}</code>	The actual value of the <code>whhttp:method</code> <i>attribute information item</i> , if present.
<code>{http:input:serialization}</code>	The actual value of the <code>whhttp:inputSerialization</code> <i>attribute information item</i> , if present; otherwise, the default value as defined in 6.4 HTTP Binding Rules .
<code>{http:output:serialization}</code>	The actual value of the <code>whhttp:outputSerialization</code> <i>attribute information item</i> , if present; otherwise, the default value as defined in 6.4 HTTP Binding Rules .
<code>{http:fault:serialization}</code>	The actual value of the <code>whhttp:faultSerialization</code> <i>attribute information item</i> , if present; otherwise "application/xml".
<code>{http:query:parameter:separator:default}</code>	The actual value of the <code>whhttp:queryParameterSeparatorDefault</code> <i>attribute information item</i> , if present; otherwise, "&".
<code>{http:query:parameter:separator}</code>	The actual value of the <code>whhttp:queryParameterSeparator</code> <i>attribute information item</i> , if present.

6.6 Declaring HTTP Headers

6.6.1 Description

HTTP allows the use of headers in messages. This binding extension allows users to declare the HTTP headers in use on a per message and on a per-fault basis.

6.6.2 Relationship to WSDL Component Model

The HTTP Header binding extension specification adds the following property to the WSDL component model (as defined in [\[WSDL 2.0 Core Language\]](#)):

- `{http:headers}` OPTIONAL. A set of [HTTP Header](#) components as defined in [6.6.3 HTTP Header component](#), to the [Binding Message Reference](#) component.
- Similarly, `{http:headers}` OPTIONAL, to the [Binding Fault](#) component.

A [Binding Message Reference](#) or a [Binding Fault](#) component's `{http:headers}` property MUST NOT contain multiple [HTTP Header](#) components with the same `{name}` property.[†]

6.6.3 HTTP Header component

An [HTTP Header](#) component describes an abstract piece of header data (HTTP header field) that is associated with the exchange of messages between the communicating parties. The presence of a [HTTP Header](#) component in a WSDL description indicates that the service support headers, and MAY require a client interacting with the service to use the described header field. Zero or one such header field may be used.

The properties of the HTTP Header component are as follows:

- {name} REQUIRED. An *xs:string* whose pattern facet is `"[/!#- '*+ \-. 0-9A-Z^_z|~]+"`, the name of the HTTP header field. The value of this property follows the `field-name` production rules as specified in section 4.2 of [ETF RFC 2616].
- {type definition} REQUIRED. A [Type Definition](#) component, in the {[type definitions](#)} property of the [Description](#) component, constraining the value of the HTTP header field. This type MUST be a simple type.[‡]
- {required} REQUIRED. An *xs:boolean* indicating if the HTTP header field is required. If the value is "true", then the HTTP header field MUST be included in the message.[‡] If it is "false", then the HTTP header field MAY be included.
- {parent} REQUIRED. The [Binding Fault](#) or [Binding Message Reference](#) component that contains this component in its {[http headers](#)} property.

6.6.4 XML Representation

```

<description>
  <binding name="xs:NCName" type="http://www.w3.org/ns/wsdL/http" >
    <fault ref="xs:QName">
      <whhttp:header name="xs:string" type="xs:QName"
        required="xs:boolean"? >
        <documentation />*
      </whhttp:header>*
      ...
    </fault>*
    <operation ref="xs:QName" >
      <input messageLabel="xs:NCName"?>
        <whhttp:header ... />*
        ...
      </input>*
      <output messageLabel="xs:NCName"?>
        <whhttp:header ... />*
        ...
      </output>*
    </operation>*
  </binding>
</description>

```

The XML representation for a [HTTP Header](#) component is an *element information item* with the following Infoset properties:

- A [local name] of header
- A [namespace name] of "http://www.w3.org/ns/wsdL/http"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *name attribute information item* with the following Infoset properties:
 - A [local name] of name
 - A [namespace name] which has no value
 - A type of *xs:string* whose pattern facet is `"[/!#- '*+ \-. 0-9A-Z^_z|~]+"`.
 - A REQUIRED *type attribute information item* with the following Infoset properties:
 - A [local name] of type
 - A [namespace name] which has no value
 - A type of *xs:QName*
 - An OPTIONAL *required attribute information item* with the following Infoset properties:
 - A [local name] of required
 - A [namespace name] which has no value
 - A type of *xs:boolean*
 - Zero or more namespace qualified *attribute information items*. The [namespace name] of such *attribute information items* MUST NOT be "http://www.w3.org/ns/wsdL" and MUST NOT be "http://www.w3.org/ns/wsdL/http".

- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* as defined in [[WSDL 2.0 Core Language](#)].
 2. Zero or more namespace-qualified *element information items* amongst its [children]. The [namespace name] of such *element information items* MUST NOT be "http://www.w3.org/ns/wsdl" and MUST NOT be "http://www.w3.org/ns/wsdl/http".

6.6.5 Mapping from XML Representation to Component Properties

See [Table 6-3](#).

Table 6-3. Mapping from XML Representation to HTTP Header component-related Properties

Property	Value
{http headers}	The set of HTTP Header components corresponding to all the header <i>element information item</i> in the [children] of the fault, input OR output <i>element information item</i> , if any.
{name}	The value of the <i>name attribute information item</i> .
{type definition}	The Type Definition component from the {type definitions} property of the Description component resolved to by the value of the <i>type attribute information item</i> .
{required}	The actual value of the <i>required attribute information item</i> , if present; otherwise "false".
{parent}	The Binding Fault or Binding Message Reference component corresponding to the fault, input OR output <i>element information item</i> in [parent].

6.6.6 IRI Identification Of An HTTP Header component

WSDL Version 2.0 Part 1: Core Language [[WSDL 2.0 Core Language](#)] defines a [fragment identifier syntax](#) for identifying components of a WSDL 2.0 document.

An [HTTP Header](#) component can be identified using the [wsdl.extension](#) XPointer Framework scheme:

```
wsdl.extension(http://www.w3.org/ns/wsdl/http, whttp.header(parent/name))
```

1. *parent* is the pointer part of the [{parent}](#) component, as [specified in WSDL Version 2.0 Part 1: Core Language](#).
2. *name* is the [{name}](#) property value.

6.7 Specifying HTTP Error Code for Faults

6.7.1 Description

For every [Interface Fault](#) component contained in an [Interface](#) component, an HTTP error code MAY be defined. It represents the error code that will be used by the service in case the fault needs to be returned.

The fault definition SHOULD agree with the definition of the HTTP error codes, as specified in section 8 of [[IETF RFC 3205](#)].[†]

6.7.2 Relationship to WSDL Component Model

The HTTP Fault binding extension adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- [{http error status code}](#) REQUIRED. A union of *xs:int* and *xs:token* where the allowed token value is "#any", to the [Binding Fault](#) component. An integer value of this property identifies the error Status-Code as defined by [[IETF RFC 2616](#)] that the service will use in

case the fault is returned.[†] If the value of this property is "#any", no claim is made by the service.

6.7.3 XML Representation

```
<description>
  <binding >
    <fault ref="xs:QName"
           whttp:code="union of xs:int, xs:token"? >
    </fault>*
  </binding>
</description>
```

The XML representation for binding an HTTP Fault is an *attribute information item* with the following Infoset properties:

- a code *OPTIONAL attribute information item*
 - A [local name] of code
 - A [namespace name] of "http://www.w3.org/ns/wsdli/http"
 - A type of union of xs:int and xs:token where the allowed token value is "#any"

6.7.4 Mapping from XML Representation to Component Properties

See [Table 6-4](#).

Table 6-4. Mapping from XML Representation to [Binding Fault](#) component Extension Properties

Property	Value
{ http error status code }	The actual value of the whttp:code <i>attribute information item</i> , if present; otherwise "#any".

6.8 Serialization Format of Instance Data

This section specifies three serialization formats defining rules to encode the [instance data](#) of an input or output message as an HTTP message. [Table 6-5](#) and [Table 6-6](#) give an overview of those serialization formats and their constraints. All of them allow serialization of parts of the [instance data](#) in the HTTP Request IRI, as defined in section [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#).

Other serialization formats may be defined. Those MAY place restrictions on the style of the [Interface Operation](#) bound.

Table 6-5. Applicability of the serialization formats defined in this section for this HTTP binding

		Serialization of the instance data in parts of an HTTP message			
		In the request URI	In the message body		
			<i>application/x-www-form-urlencoded</i>	<i>multipart/form-data</i>	<i>application/xml</i>
HTTP request (input message)	Without message body: GET, DELETE, ...	All, some or none	-	-	-
	With message body: POST, PUT, ...	All, some or none	Remainder	All	All
HTTP response (output message)		-	-	-	All

Table 6-6. Operation styles required for using serialization formats defined below as input serialization

HTTP Method	Request			
	Request URI: query parameters or path components	Input serialization		
		<i>application/x-www-form-urlencoded</i>	<i>multipart/form-data</i>	<i>application/xml</i>
Without message body: GET, DELETE, ...	IRI style	IRI style	-	-
With message body: POST, PUT, ...	IRI style, if any data is serialized as path components or query parameters	IRI style	Multipart style	None required

6.8.1 Serialization of the instance data in parts of the HTTP request IRI

This section defines templating rules for the `{http location}` property of the [Binding Operation](#) component. Templating is used by the serialization formats defined in section [6.8 Serialization Format of Instance Data](#), and MAY be reused by other serialization formats.

With this HTTP binding, part of the instance data for HTTP requests MAY be serialized in the HTTP request IRI, and another part MAY be serialized in the HTTP message body.

If the `{style}` property of the [Interface Operation](#) bound has a value of "http://www.w3.org/ns/wSDL/style/iri" as defined in [4.2 IRI Style](#), and if the `{http location}` property of the [Binding Operation](#) component is present, the value of the `{http location}` property component is used as a template[‡] which is combined with the `{address}` property of the endpoint element to form the full IRI to be used in an HTTP request, as specified in section [6.5.2 Relationship to WSDL Component Model](#).

The resulting IRI MUST be mapped to an URI for use in the HTTP Request as per section 3.1 "Mapping of IRIs to URIs" of the IRI specification [[IETF RFC 3987](#)].[‡] Additional rules for the serialization of the HTTP request IRI MAY be defined by a serialization format.

6.8.1.1 Construction of the request IRI using the `{http location}` property

The `{http location}` property MAY cite local names of elements from the [instance data](#) of the message to be serialized in request IRI. Citing is performed:

- either by enclosing the element name within curly braces. For example, "temperature/{town}". See [Example 6-1](#) for additional details;
- or by enclosing the element name within exclamated-curly braces, to include the element without percent-encoding. For example, "temperature/{!town}". Detailed rules follow.

The `{http location}` property MUST conform to the following EBNF [[ISO/IEC 14977:1996](#)] grammar, which represents the patterns for constructing the request IRI:[‡]

```

httpLocation ::= charData? (( openBrace | closeBrace | template ) charData?)*
charData ::= [^{}]*
openBrace ::= '{'
closeBrace ::= '}'
template ::= rawTemplate | encodedTemplate
rawTemplate ::= '{!' NCName '}'
encodedTemplate ::= '{' NCName '}'

```

The request IRI is constructed as follows (ALPHA and DIGIT below are defined as per [[IETF RFC 4234](#)]):

- The local name in a template SHOULD match at least one element from the [instance data](#) of the input message.[‡] When there is no match, the template is replaced by an empty string. Otherwise, the template consumes the first non-consumed matching element from the [instance data](#). The next occurrence of the template consumes the next non-consumed matching element, and so on until all templates are processed. Matching elements are consumed in the order in which they appear in the [instance data](#). Cited elements (i.e. elements referenced in templates) MUST NOT carry an `xs:nil` attribute whose value is `"true"`[‡].
- Each raw template (`rawTemplate` production in the grammar above) is replaced by the possibly empty single value of the corresponding element from the [instance data](#). No percent-encoding is performed.
- Each encoded template (`encodedTemplate` production in the grammar above) NOT preceded in the `{http:location}` property by a "?" character is replaced by the possibly empty single value of the corresponding element from the [instance data](#). Encoding is performed as follows:
 - The characters in the range: "&" | ";" | "!" | "\$" | "'" | "(" | ")" | "*" | "+" | "," | "=" | ":" | "@" SHOULD be percent-encoded.
 - The other characters, EXCEPT the ones in the range: ALPHA | DIGIT | "-" | "." | "_" | "~", MUST be percent-encoded.
- Each encoded template (`encodedTemplate` production in the grammar above) preceded in the `{http:location}` property by a "?" character is replaced by the possibly empty single value of the corresponding element from the [instance data](#). Encoding is performed as follows:
 - The value of the `{http:query:parameter:separator}` property, if present; otherwise the value of the `{http:query:parameter:separator:default}` property, MUST be percent-encoded.
 - The characters in the range: "&" | ";" | "!" | "\$" | "'" | "(" | ")" | "*" | "+" | "," | "=" | ":" | "@" | "?" | "/" SHOULD be percent-encoded.
 - The other characters, EXCEPT the ones in the range: ALPHA | DIGIT | "-" | "." | "_" | "~", MUST be percent-encoded.
- Each uncited element (i.e. each element not referenced in a template) to be serialized, if any, is encoded as for an encoded template.
- Percent-encoding MUST be performed using the UTF-8 representation of the character as prescribed by section 6.4 of [[IETF RFC 3987](#)].
- Each double curly brace (`openBrace` or `closeBrace` production in the grammar above) is replaced by a single literal curly brace ("{" or "}" respectively). This provides a simple escaping mechanism.

Note that the mechanism described in this section could be used to indicate the entire absolute IRI, including the scheme, host, or port, for example:

```
{scheme}://{host}:{port}/temperature/{town}
```

or even:

```
{!myIRI}
```

6.8.2 Serialization as "application/x-www-form-urlencoded"

This serialization format is designed to allow a client or Web service to produce an IRI based on the [instance data](#) of a message and serialize a query string in the HTTP message body as `application/x-www-form-urlencoded`.

If this format is used then the `{style}` property of [Interface Operation](#) component being bound MUST contain a value of `"http://www.w3.org/ns/wsd/style/iri"` as defined in [4.2 IRI Style](#), i.e. this serialization format may only be used to serialize the HTTP request corresponding to the initial message of an interface operation.[‡]

For the HTTP binding defined in this section ([6. WSDL HTTP Binding Extension](#)), `"application/x-www-form-urlencoded"` MAY be used as a [serialization format](#) for an input

message (HTTP Request), but MUST NOT be used as a [serialization format](#) for an output or fault message (HTTP Response).[‡]

6.8.2.1 Case of elements cited in the {http location} property

In this serialization, the rules for constructing the HTTP request IRI using elements cited in the {[http location](#)} property defined in [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#) apply. Additional rules for constructing the HTTP request IRI follow.

6.8.2.2 Serialization of content of the instance data not cited in the {http location} property

If not all elements from the [instance data](#) are cited in the {[http location](#)} property, or if the property is not present on the [Binding Operation](#) component, then additional serialization rules apply.[‡]

The remainder of the instance data is formatted as a query string as defined in [6.8.2.2.1 Construction of the query string](#).

If the HTTP method used for the request does not allow a message body, then this query string is serialized as parameters in the request IRI (see [6.8.2.2.3 Serialization in the request IRI](#)), otherwise it is serialized in the message body (see [6.8.2.2.4 Serialization in the message body](#)).

6.8.2.2.1 CONSTRUCTION OF THE QUERY STRING

For elements of the instance data not cited in the {[http location](#)} property, a query string is constructed as follows.[‡]

Non-nil elements with a possibly empty single value of the [instance data](#) not cited are serialized as query parameters in the order they appear in the instance data.

The [instance data](#) MUST NOT contain elements with an `xs:nil` attribute whose value is "true".[‡] Each parameter pair is separated by the value of the {[http query parameter separator](#)} property, if present, or the value of the {[http query parameter separator default](#)} property.

- Uncited elements with single values (non-list) are serialized as a single name-value parameter pair. The name of the parameter is the local name of the uncited element, and the value of the parameter is the value of the uncited element.
- Uncited elements with list values are serialized as one name-value parameter pair per-list value. The name of each parameter is the local name of the uncited element, and the value of each parameter is the corresponding value in the list. The order of the list values is preserved.
- Replacement values falling outside the range (ALPHA and DIGIT below are defined as per [[IETF RFC 4234](#)]): ALPHA | DIGIT | "-" | "." | "_" | "~" | "!" | "\$" | "&" | "'" | "(" | ")" | "*" | "+" | "," | ";" | "=" | ":" | "@", MUST be percent-encoded. Percent-encoding MUST be performed using the UTF-8 representation of the character as prescribed by section 6.4 of [[IETF RFC 3987](#)].

Example 6-1. Query string generation

The following instance data of an input message:

```
<data>
  <town>Fréjus</town>
  <date>2007-06-26</date>
  <unit>C</unit>
</data>
```

with the following value of the {[http location](#)} property:

```
'temperature/{town}'
```

and the following value of the [{http_query_parameter_separator default}](#) property:

'&'

will produce the following query string:

date=2007-06-26&unit=C

6.8.2.2.2 CONTROLLING THE SERIALIZATION OF THE QUERY STRING IN THE REQUEST IRI

This serialization format adds the following property to the [Binding Operation](#) component:

- [{http location ignore uncited}](#) REQUIRED. A *xs:boolean*. This boolean indicates whether elements not cited in the [{http location}](#) property MUST be appended to the request IRI or ignored. If the value of this property is "false", the rules defined in section [6.8.2.2.3 Serialization in the request IRI](#) dictate how to serialize elements not cited in [{http location}](#) in the request IRI. Otherwise, those are NOT serialized in the request IRI.

When serializing an HTTP request that does not allow an HTTP message body, and when [{http location ignore uncited}](#) is "true", any element NOT cited in the [{http location}](#) property MUST be defined in the schema as *nillable*, or have a *default* value, or appear no less frequently than specified by the *minOccurs* value. The element declaration SHOULD NOT combine a default value with *nillable*.[†]

The XML representation for this property is an *attribute information item* with the following Infoset properties:

- An OPTIONAL *ignoreUncited* *attribute information item* with the following Infoset properties:
 - A [local name] of *ignoreUncited*
 - A [namespace name] of "http://www.w3.org/ns/wsd/1/http"
 - A type of *xs:boolean*

The mapping from the XML representation to component properties is as follows:

Table 6-7. Mapping from XML Representation to Binding Operation component Extension Properties

Property	Value
{http location ignore uncited}	The actual value of the <i>http:ignoreUncited</i> <i>attribute information item</i> , if present; otherwise, "false".

6.8.2.2.3 SERIALIZATION IN THE REQUEST IRI

If the HTTP request method used does not allow HTTP message body (e.g. "GET" and "DELETE"), and if the value of the [{http location ignore uncited}](#) property is "false", then the following rules apply.[†]

If the [{http location}](#) property is not present, or if it is present and its value does not contain a "?" (question mark) character, a "?" is appended to the request IRI. If it does already contain a question mark character, then the value of the [{http_query_parameter_separator}](#) property, if present, or the value of the [{http_query_parameter_separator default}](#) property otherwise, is appended.

Finally, the query string computed in [6.8.2.2.1 Construction of the query string](#) is appended.

Example 6-2. Instance data serialized in an IRI

The instance data defined in [Example 6-1](#) with the following operation declaration:

```
<operation ref='t:data'
  whttp:location='temperature/{town}'
  whttp:method='GET' />
```

and the following endpoint declaration:

```
<endpoint name='e' binding='t:b'
  address='http://ws.example.com/service1/' />
```

will serialize the message in the HTTP request as follows:

```
GET http://ws.example.com/service1/temperature/Fr%C3%A9jus?date=2007-06-26&unit=C HTTP/1.1
Host: ws.example.com
```

6.8.2.2.4 SERIALIZATION IN THE MESSAGE BODY

If the HTTP request method used does allow an HTTP message body (e.g. "POST" and "PUT"), then the following rules apply.[‡]

Finally, the query string computed in [6.8.2.2.1 Construction of the query string](#) is used as the value of the HTTP message body.

The Content-Type HTTP header field must have the value `application/x-www-form-urlencoded`.[‡]

Example 6-3. Instance data serialized in the HTTP Request IRI and message body

The instance data defined in [Example 6-1](#) with the following operation declaration:

```
<operation ref='t:data'
  whttp:inputSerialization='application/x-www-form-urlencoded'
  whttp:location='temperature/{town}'
  whttp:method='POST' />
```

and the following endpoint declaration:

```
<endpoint name='e' binding='t:b'
  address='http://ws.example.com/service1/' />
```

will serialize the message in the HTTP request as follow:

```
POST http://ws.example.com/service1/temperature/Fr%C3%A9jus HTTP/1.1
Host: ws.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: ...

date=2007-06-26&unit=C
```

6.8.3 Serialization as "application/xml"

In this serialization, for HTTP requests, the rules for constructing the HTTP request IRI defined in [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#) apply if the `{style}` property of the [Interface Operation](#) bound has a value of "http://www.w3.org/ns/wsd/1.0/iri" as defined in [4.2 IRI Style](#).

The [instance data](#) of the input, output or fault message is serialized as an XML document in the message body of the HTTP message, following the serialization defined in [\[Canonical XML\]](#). Therefore, it is only suitable for HTTP requests using methods allowing message bodies (i.e., for the HTTP binding defined in this specification, input messages where the HTTP method selected has a body), and for HTTP responses (i.e. output and fault messages for the HTTP binding defined in this specification).

The Content-Type HTTP header MUST have the value `application/xml` [\[IETF RFC 3023\]](#), or a media type compatible with `application/xml` as specified in section [6.4.3.1 Serialization rules for XML messages](#).[‡] Other HTTP headers MAY be used.

6.8.4 Serialization as "multipart/form-data"

In this serialization, for HTTP requests, the rules for constructing the HTTP request IRI defined in [6.8.1 Serialization of the instance data in parts of the HTTP request IRI](#) apply if the {*style*} property of the [Interface Operation](#) bound has a value of "http://www.w3.org/ns/wsdl/style/iri" as defined in [4.2 IRI Style](#).

This format is for legacy compatibility to permit the use of XForms clients with [*ietf RFC 2388*] servers. This serialization format may only be used when binding [Interface Operation](#) components whose {*style*} property has a value of "http://www.w3.org/ns/wsdl/style/multipart" as defined in [4.3 Multipart style](#), i.e. this serialization format may only be used to serialize the HTTP request corresponding to the initial message of an interface operation.[‡]

Specifically, for the HTTP binding defined in this section ([6. WSDL HTTP Binding Extension](#)), "multipart/form-data" MAY be used as a [serialization format](#) for an input message (HTTP Request), but MUST NOT be used as a [serialization format](#) for an output or fault message (HTTP Response).[‡] This format serializes the instance data in the HTTP message body, making it only suitable for HTTP requests using methods allowing message bodies.

Each element in the sequence is serialized into a part as follow:

1. The Content-Disposition header MUST have the value form-data, and its name parameter is the local name of the element.[‡]
2. The Content-Type header MUST have the value:[‡]
 - application/xml (or a media type compatible with application/xml) if the element has a complex type;
 - application/octet-stream if the element is of type xs:base64Binary, xs:hexBinary, or a derived type;
 - text/plain if the element has a simple type; The charset MUST be set appropriately. UTF-8 or UTF-16 MUST be at least supported.
3. If the type is xs:base64Binary, xs:hexBinary, xs:anySimpleType or a derived type, the content of the part is the content of the element. If the type is a complex type, the element is serialized following the rules defined in the [6.8.3 Serialization as application/xml](#).

The [instance data](#) MUST NOT contain elements with an xs:nil attribute whose value is "true".[‡]

Example 6-4. Example of multipart/form-data

The following instance data of an input message:

```
<data>
  <town>
    <name>Fréjus</name>
    <country>France</country>
  </town>
  <date>2007-06-26</date>
</data>
```

with the following operation element:

```
<operation ref='t:data'
  whttp:location='temperature'
  whttp:method='POST'
  whttp:inputSerialization='multipart/form-data' />
```

will serialize the message as follow:

```
Content-Type: multipart/form-data; boundary=AaB03x
Content-Length: xxx

--AaB03x
Content-Disposition: form-data; name="town"
Content-Type: application/xml

<town>
  <name>Fréjus</name>
  <country>France</country>
</town>
--AaB03x
Content-Disposition: form-data; name="date"
```

```
Content-Type: text/plain; charset=utf-8
```

```
2007-06-26
--AaB03x--
```

6.9 Specifying the Content Encoding

6.9.1 Description

Every [Binding Message Reference](#) and [Binding Fault](#) component MAY indicate which content encodings, as defined in section 3.5 of [[IETF RFC 2616](#)], are available for this particular message.

The HTTP binding extension provides a mechanism for indicating a default value at the [Binding](#) component and [Binding Operation](#) levels.

If no value is specified, no claim is being made.

6.9.2 Relationship to WSDL Component Model

The HTTP binding extension specification adds the following properties to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- {http content encoding default} OPTIONAL. A *xs:string* to the [Binding](#) component. This property indicates the default content encodings available for all [Binding Message Reference](#) and [Binding Fault](#) components of this [Binding](#).
- {http content encoding default} OPTIONAL. A *xs:string* to the [Binding Operation](#) component. This property indicates the default content encodings available for all [Binding Message Reference](#) of this [Binding Operation](#).
- {http content encoding} OPTIONAL. A *xs:string* to the [Binding Message Reference](#) component. This property indicates the content encodings available for this [Binding Message Reference](#) component. If this property does not have a value, the value of the {http content encoding default} property of the parent [Binding Operation](#) component is used instead. If that itself has no value, the value from the [Binding Operation](#) component's parent [Binding](#) component is used instead.
- Similarly, {http content encoding} OPTIONAL, to the [Binding Fault](#) component

These properties are not relevant when HTTP 1.0 is used.

6.9.3 XML Representation

```
<description>
  <binding name="xs:NCName" interface="xs:QName"? type="xs:anyURI"
    whttp:contentEncodingDefault="xs:string"? >

    <fault ref="xs:QName"
      whttp:contentEncoding="xs:string"? >
    </fault>*

    <operation location="xs:anyURI"?
      whttp:contentEncodingDefault="xs:string"? >
      <input messageLabel="xs:NCName"?
        whttp:contentEncoding="xs:string"? />

      <output messageLabel="xs:NCName"?
        whttp:contentEncoding="xs:string"? />

    </operation>
  </binding>
</description>
```

The XML representation for specifying the content encoding is an OPTIONAL *attribute information item* for the input, output, and fault *element information items* with the following Infoset properties:

- A [local name] of `contentEncoding`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/http"`
- A type of `xs:string`

The XML representation for specifying the default content encoding is an OPTIONAL *attribute information item* for the binding *element information item* or binding's child operation *element information items* with the following Infoset properties:

- A [local name] of `contentEncodingDefault`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/http"`
- A type of `xs:string`

6.9.4 Mapping from XML Representation to Component Properties

See [Table 6-8](#).

Table 6-8. Mapping from XML Representation to [Interface Message Reference](#) component Extension Properties

Property	Value
<code>{http content encoding default}</code> of the Binding component	The actual value of the <code>whhttp:contentEncodingDefault</code> <i>attribute information item</i> of the binding <i>element information item</i> , if present.
<code>{http content encoding default}</code> of the Binding Operation component	The actual value of the <code>whhttp:contentEncodingDefault</code> <i>attribute information item</i> of the operation <i>element information item</i> , if present.
<code>{http content encoding}</code> of the Binding Message Reference component	The actual value of the <code>whhttp:contentEncoding</code> <i>attribute information item</i> of the input or output <i>element information item</i> , if present.
<code>{http content encoding}</code> of the Binding Fault component	The actual value of the <code>whhttp:contentEncoding</code> <i>attribute information item</i> of the fault <i>element information item</i> , if present.

6.10 Specifying the Use of HTTP Cookies

6.10.1 Description

The `{http cookies}` property allows [Binding](#) components to indicate that HTTP cookies (as defined by [[IETF RFC 2965](#)]) are used by specific operations of the interface that this binding applies to.

6.10.2 Relationship to WSDL Component Model

The HTTP binding extension specification adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- `{http cookies}` REQUIRED. A `xs:boolean` to the [Binding](#) component.

6.10.3 XML Representation

```
<description>
  <binding name="xs:NCName" interface="xs:QName"? type="xs:anyURI"
    whhttp:cookies="xs:boolean"? >
  </binding>
</description>
```

The XML representation for specifying the use of HTTP cookies is an OPTIONAL *attribute information item* with the following Infoset properties:

- A [local name] of `cookies`
- A [namespace name] of `"http://www.w3.org/ns/wsdl/http"`
- A type of `xs:boolean`

6.10.4 Mapping from XML Representation to Component Properties

See [Table 6-9](#).

Table 6-9. Mapping from XML Representation to Binding component Extension Properties

Property	Value
<code>{http cookies}</code>	The actual value of the <code>whhttp:cookies</code> <i>attribute information item</i> ; otherwise, "false". A value of "true" means that the service relies on cookies and that the client MUST understand them. [‡]

6.11 Specifying HTTP Access Authentication

6.11.1 Description

Every [Endpoint](#) component MAY indicate the use of an HTTP access authentication mechanism (as defined by [[IETF RFC 2616](#)]) for the endpoint described.

This binding extension specification allows the authentication scheme and realm to be specified.

6.11.2 Relationship to WSDL Component Model

The HTTP binding extension specification adds the following property to the WSDL component model (as defined in [[WSDL 2.0 Core Language](#)]):

- `{http authentication scheme}` OPTIONAL. A `xs:token` with one of the values "basic" or "digest", to the [Endpoint](#) component, corresponding to the HTTP authentication scheme used. When present, this property indicates the authentication scheme in use: "basic" indicates the Basic Access Authentication scheme defined in [[IETF RFC 2617](#)], and "digest" indicates the Digest Access Authentication scheme as defined in [[IETF RFC 2617](#)].
- `{http authentication realm}` OPTIONAL. A `xs:string` to the [Endpoint](#) component. It corresponds to the realm authentication parameter defined in [[IETF RFC 2617](#)]. If the `{http authentication scheme}` property is present, then this property **MUST** be present.[‡]

6.11.3 XML Representation

```
<description>
  <service>
    <endpoint name="xs:NCName" binding="xs:QName" address="xs:anyURI"? >
      whhttp:authenticationScheme="xs:token"?
      whhttp:authenticationRealm="xs:string"? />
    </endpoint>
  </service>
</description>
```

The XML representation for specifying the use of HTTP access authentication is two OPTIONAL *attribute information items* with the following Infoset properties:

- An OPTIONAL `authenticationScheme` *attribute information item* with the following Infoset properties:
 - A [local name] of `authenticationScheme`
 - A [namespace name] of `"http://www.w3.org/ns/wsdl/http"`
 - A type of `xs:token` where the allowed token values are "basic" and "digest".

- An OPTIONAL `authenticationRealm` *attribute information item* with the following Infoset properties:
 - A [local name] of `authenticationRealm`
 - A [namespace name] of `"http://www.w3.org/ns/wsd/11/http"`
 - A type of `xs:string`

6.11.4 Mapping from XML Representation to Component Properties

See [Table 6-10](#).

Table 6-10. Mapping from XML Representation to Endpoint component Extension Properties

Property	Value
<code>{http authentication scheme}</code>	The actual value of the <code>whhttp:authenticationScheme</code> <i>attribute information item</i> , if present.
<code>{http authentication realm}</code>	The actual value of the <code>whhttp:authenticationRealm</code> <i>attribute information item</i> , if present; otherwise, if the <code>whhttp:authenticationScheme</code> <i>attribute information item</i> is present, "" (the empty value).

6.12 Conformance

An *element information item*, whose namespace name is `"http://www.w3.org/ns/wsd/11/http"` and whose local part is `description`, conforms to this binding extension specification if: the *element information items* and *attribute information items*, whose namespace is `http://www.w3.org/ns/wsd/11/http`, conform to the XML Schema for that element or attribute, as defined by this specification and, additionally, adheres to all the constraints contained in this specification.

7. References

7.1 Normative References

[ISO/IEC 14977:1996]

[Extended BNF](#), ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission), Dec 1996. Available at http://isotc.iso.org/livelink/livelink/fetch/2000/2489/1/ttf_Home/PubliclyAvailableStandards.htm.

[IETF RFC 2119]

[Key words for use in RFCs to Indicate Requirement Levels](#), S. Bradner, Author. Internet Engineering Task Force, March 1997. Available at <http://www.ietf.org/rfc/rfc2119.txt>.

[IETF RFC 2388]

[Returning Values from Forms: multipart/form-data](#), L. Masinter, Author. Internet Engineering Task Force, August 1998. Available at <http://www.ietf.org/rfc/rfc2388.txt>.

[IETF RFC 2616]

[Hypertext Transfer Protocol -- HTTP/1.1](#), R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, T. Berners-Lee, Authors. Internet Engineering Task Force, June 1999. Available at <http://www.ietf.org/rfc/rfc2616.txt>.

[IETF RFC 2617]

[HTTP Authentication: Basic and Digest Access Authentication](#), J. Franks, P. Hallam-Baker, J. Hostetler, S. Lawrence, P. Leach, A. Luotonen, L. Stewart, June 1999. Available at <http://www.ietf.org/rfc/rfc2616.txt>.

[IETF RFC 2818]

[HTTP Over TLS](#), E. Rescorla, Author. Internet Engineering Task Force, May 2000. Available at <http://www.ietf.org/rfc/rfc2818.txt>.

[IETF RFC 2965]

[HTTP State Management Mechanism](#), D. Kristol, L. Montulli Authors. Internet Engineering Task Force, October 2000. Available at <http://www.ietf.org/rfc/rfc2965.txt>.

[IETF RFC 3023]

[XML Media Types](#), M. Murata, S. St. Laurent, D. Kohn, Authors. Internet Engineering Task Force, January 2001. Available at <http://www.ietf.org/rfc/rfc3023.txt>.

[IETF RFC 3205]

[On the use of HTTP as a Substrate](#), K. Moore, Authors. Internet Engineering Task Force, February 2002. Available at <http://www.ietf.org/rfc/rfc3205.txt>.

[IETF RFC 3986]

[Uniform Resource Identifiers \(URI\): Generic Syntax](#), T. Berners-Lee, R. Fielding, L. Masinter, Authors. Internet Engineering Task Force, January 2005. Available at <http://www.ietf.org/rfc/rfc3986.txt>.

[IETF RFC 3987]

[Internationalized Resource Identifiers \(IRIs\)](#), M. Duerst, M. Suignard, Authors. Internet Engineering Task Force, January 2005. Available at <http://www.ietf.org/rfc/rfc3987.txt>.

[IETF RFC 4234]

[Augmented BNF for Syntax Specifications: ABNF](#), D. Crocker, P. Overell, Authors. Internet Engineering Task Force, October 2005. Available at <http://www.ietf.org/rfc/rfc4234.txt>.

[Web Architecture]

[Architecture of the World Wide Web, Volume One](#), I. Jacobs, and N. Walsh, Editors. World Wide Web Consortium, 15 December 2004. This version of the "Architecture of the World Wide Web, Volume One" Recommendation is <http://www.w3.org/TR/2004/REC-webarch-20041215/>. The [latest version of "Architecture of the World Wide Web, Volume One"](#) is available at <http://www.w3.org/TR/webarch/>.

[Web Services Architecture]

[Web Services Architecture](#), David Booth, Hugo Haas, Francis McCabe, Eric Newcomer, Michael Champion, Chris Ferris, David Orchard, Editors. World Wide Web Consortium, 11 February 2004. This version of the "Web Services Architecture" Working Group Note is <http://www.w3.org/TR/2004/NOTE-ws-arch-20040211/>. The [latest version of "Web Services Architecture"](#) is available at <http://www.w3.org/TR/ws-arch/>.

[WSDL 2.0 Core Language]

[Web Services Description Language \(WSDL\) Version 2.0 Part 1: Core Language](#), R. Chinnici, J.-J. Moreau, A. Ryman, S. Weerawarana, Editors. World Wide Web Consortium, 26 June 2007. This version of the "Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language" Recommendation is available at <http://www.w3.org/TR/2007/REC-wsdl20-20070626>. The [latest version of "Web Services Description Language \(WSDL\) Version 2.0 Part 1: Core Language"](#) is available at <http://www.w3.org/TR/wsdl20>.

[SOAP 1.2 Part 1: Messaging Framework (Second Edition)]

[SOAP Version 1.2 Part 1: Messaging Framework \(Second Edition\)](#), M. Gudgin, et al., Editors. World Wide Web Consortium, 24 June 2003, revised 27 April 2007. This version of the "SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)" Recommendation is <http://www.w3.org/TR/2007/REC-soap12-part1-20070427/>. The [latest version of "SOAP Version 1.2 Part 1: Messaging Framework"](#) is available at <http://www.w3.org/TR/soap12-part1/>.

[SOAP 1.2 Part 2: Adjuncts (Second Edition)]

[SOAP Version 1.2 Part 2: Adjuncts \(Second Edition\)](#), M. Gudgin, et al., Editors. World Wide Web Consortium, 24 June 2006, revised 27 April 2007. This version of the "SOAP Version 1.2 Part 2: Adjuncts (Second Edition)" Recommendation is <http://www.w3.org/TR/2007/REC-soap12-part2-20070427/>. The [latest version of "SOAP Version 1.2 Part 2: Adjuncts"](#) is available at <http://www.w3.org/TR/soap12-part2/>.

[XML 1.0]

[Extensible Markup Language \(XML\) 1.0 \(Fourth Edition\)](#), T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, Editors. World Wide Web Consortium, 10 February 1998, revised 16 August 2006. This version of the XML 1.0 Recommendation is <http://www.w3.org/TR/2006/REC-xml-20060816/>. The [latest version of "Extensible Markup Language \(XML\) 1.0"](#) is available at <http://www.w3.org/TR/REC-xml>.

[Canonical XML]

[Canonical XML](#), J. Boyer, Author. World Wide Web Consortium, 15 March 2001. This version of the Canonical XML Recommendation is <http://www.w3.org/TR/2001/REC-xml-c14n-20010315>. The [latest version of Canonical XML](#) is available at <http://www.w3.org/TR/xml-c14n>.

[XML Information Set]

[XML Information Set \(Second Edition\)](#), J. Cowan and R. Tobin, Editors. World Wide Web Consortium, 24 October 2001, revised 4 February 2004. This version of the XML

Information Set Recommendation is <http://www.w3.org/TR/2004/REC-xml-infoset-20040204>. The [latest version of XML Information Set](#) is available at <http://www.w3.org/TR/xml-infoset>.

[XML Schema Structures]

[XML Schema Part 1: Structures Second Edition](#), H. Thompson, D. Beech, M. Maloney, and N. Mendelsohn, Editors. World Wide Web Consortium, 2 May 2001, revised 28 October 2004. This version of the XML Schema Part 1 Recommendation is <http://www.w3.org/TR/2004/REC-xmlschema-1-20041028>. The [latest version of XML Schema Part 1](#) is available at <http://www.w3.org/TR/xmlschema-1>.

[XML Schema Datatypes]

[XML Schema Part 2: Datatypes Second Edition](#), P. Byron and A. Malhotra, Editors. World Wide Web Consortium, 2 May 2001, revised 28 October 2004. This version of the XML Schema Part 2 Recommendation is <http://www.w3.org/TR/2004/REC-xmlschema-2-20041028>. The [latest version of XML Schema Part 2](#) is available at <http://www.w3.org/TR/xmlschema-2>.

[XForms 1.0]

[XForms 1.0 \(Second Edition\)](#), J. Boyer, et al., Editors. World Wide Web Consortium, 14 October 2003, revised 14 March 2006. This version of the XForms 1.0 Recommendation is <http://www.w3.org/TR/2006/REC-xforms-20060314/>. The [latest version of XForms 1.0](#) is available at <http://www.w3.org/TR/xforms/>.

7.2 Informative References

[WSA 1.0 Core]

[Web Services Addressing 1.0 - Core](#), M. Gudgin, M. Hadley, T. Rogers, Editors. World Wide Web Consortium, 9 May 2006. This version of Web Services Addressing 1.0 - Core Recommendation is <http://www.w3.org/TR/2006/REC-ws-addr-core-20060509/>. The latest version of the "Web Services Addressing 1.0 - Core" document is available from <http://www.w3.org/TR/ws-addr-core>.

[WSDL 2.0 Primer]

[Web Services Description Language \(WSDL\) Version 2.0 Part 0: Primer](#), D. Booth, C.K. Liu, Editors. World Wide Web Consortium, 26 June 2007. This version of the "Web Services Description Language (WSDL) Version 2.0 Part 0: Primer" Recommendation is available at <http://www.w3.org/TR/2007/REC-wsdl20-primer-20070626>. The [latest version of "Web Services Description Language \(WSDL\) Version 2.0 Part 0: Primer"](#) is available at <http://www.w3.org/TR/wsdl20-primer>.

[WSDL 2.0 Additional MEPs]

[Web Services Description Language \(WSDL\) Version 2.0: Additional MEPs](#), A. Lewis, Editors. World Wide Web Consortium, 26 June 2007. This version of the "Web Services Description Language (WSDL) Version 2.0: Additional MEPs" Working Group Note is available at <http://www.w3.org/TR/2007/NOTE-wsdl20-additional-meps-20070626>. The [latest version of "Web Services Description Language \(WSDL\) Version 2.0: Additional MEPs"](#) is available at <http://www.w3.org/TR/wsdl20-additional-meps>.

[SOAP Message Transmission Optimization Mechanism]

[SOAP Message Transmission Optimization Mechanism](#), N. Mendelsohn, M. Nottingham, and H. Ruellan, Editors. World Wide Web Consortium, W3C Recommendation, 25 January 2005. This version of SOAP Message Transmission Optimization Mechanism is <http://www.w3.org/TR/2005/REC-soap12-mtom-20050125/>. The [latest version of the "SOAP Message Transmission Optimization Mechanism" document](#) is available from <http://www.w3.org/TR/soap12-mtom/>.

[XPointer]

[XPointer Framework](#), Paul Grosso, Eve Maler, Jonathan Marsh, Norman Walsh, Editors. World Wide Web Consortium, 25 March 2003. This version of the XPointer Framework Proposed Recommendation is <http://www.w3.org/TR/2003/REC-xptr-framework-20030325/>. The [latest version of XPointer Framework](#) is available at <http://www.w3.org/TR/xptr-framework/>.

A. Acknowledgements (Non-Normative)

This document is the work of the [W3C Web Service Description Working Group](#).

Members of the Working Group are (at the time of writing, and by alphabetical order): Charlton Barreto (Adobe Systems, Inc), Allen Brookes (Rogue Wave Software), Dave Chappell (Sonic Software), Helen Chen (Agfa-Gevaert N. V.), Roberto Chinnici (Sun Microsystems), Kendall Clark (University of Maryland), Glen Daniels (Sonic Software), Paul Downey (British Telecommunications), Youenn Fablet (Canon), Ram Jeyaraman (Microsoft), Tom Jordahl (Adobe Systems), Anish Karmarkar (Oracle Corporation), Jacek Kopecky (DERI Innsbruck at the Leopold-Franzens-Universität Innsbruck, Austria), Amelia Lewis (TIBCO Software, Inc.), Philippe Le Hegaret (W3C), Michael Liddy (Education.au Ltd.), Kevin Canyang Liu (SAP AG), Jonathan Marsh (WSO2), Monica Martin (Sun Microsystems), Josephine Micallef (SAIC - Telcordia Technologies), Jeff Mischkinsky (Oracle Corporation), Dale Moberg (Cyclone Commerce), Jean-Jacques Moreau (Canon), David Orchard (BEA Systems, Inc.), Gilbert Pilz (BEA Systems, Inc.), Tony Rogers (Computer Associates), Arthur Ryman (IBM), Adi Sakala (IONA Technologies), Michael Shepherd (Xerox), Asir Vedamuthu (Microsoft Corporation), Sanjiva Weerawarana (WSO2), Ümit Yalçınalp (SAP AG), Peter Zehler (Xerox).

Previous members were: Eran Chinthaka (WSO2), Mark Nottingham (BEA Systems, Inc.), Hugo Haas (W3C), Vivek Pandey (Sun Microsystems), Bijan Parsia (University of Maryland), Lily Liu (webMethods, Inc.), Don Wright (Lexmark), Joyce Yang (Oracle Corporation), Daniel Schutzer (Citigroup), Dave Solo (Citigroup), Stefano Pogliani (Sun Microsystems), William Stumbo (Xerox), Stephen White (SeeBeyond), Barbara Zengler (DaimlerChrysler Research and Technology), Tim Finin (University of Maryland), Laurent De Teneuille (L'Echangeur), Johan Pauhlsson (L'Echangeur), Mark Jones (AT&T), Steve Lind (AT&T), Sandra Swearingen (U.S. Department of Defense, U.S. Air Force), Philippe Le Hégarret (W3C), Jim Hendler (University of Maryland), Dietmar Gaertner (Software AG), Michael Champion (Software AG), Don Mullen (TIBCO Software, Inc.), Steve Graham (Global Grid Forum), Steve Tuecke (Global Grid Forum), Michael Mahan (Nokia), Bryan Thompson (Hicks & Associates), Ingo Melzer (DaimlerChrysler Research and Technology), Sandeep Kumar (Cisco Systems), Alan Davies (SeeBeyond), Jacek Kopecky (Systinet), Mike Ballantyne (Electronic Data Systems), Mike Davoren (W. W. Grainger), Dan Kulp (IONA Technologies), Mike McHugh (W. W. Grainger), Michael Mealling (Verisign), Waqar Sadiq (Electronic Data Systems), Yaron Goland (BEA Systems, Inc.), Ümit Yalçınalp (Oracle Corporation), Peter Madziak (Agfa-Gevaert N. V.), Jeffrey Schlimmer (Microsoft Corporation), Hao He (The Thomson Corporation), Erik Ackerman (Lexmark), Jerry Thrasher (Lexmark), Prasad Yendluri (webMethods, Inc.), William Vambenepe (Hewlett-Packard Company), David Booth (W3C), Sanjiva Weerawarana (IBM), Asir Vedamuthu (webMethods, Inc.), Igor Sedukhin (Computer Associates), Martin Gudgin (Microsoft Corporation), Rebecca Bergersen (IONA Technologies), Ugo Corda (SeeBeyond).

The people who have contributed to [discussions on www-ws-desc@w3.org](http://www-ws-desc@w3.org) are also gratefully acknowledged.

B. Component Summary (Non-Normative)

[Table B-1](#) lists all the components in the WSDL 2.0 Adjuncts abstract Component Model, and all their properties.

Table B-1. Summary of WSDL 2.0 Adjuncts Components and their Properties

Component	Defined Properties
Binding	{http content encoding default} , {http cookies} , {http method default} , {http query parameter separator default} , {soap mep default} , {soap modules} , {soap underlying protocol} , {soap version}
Binding Fault	{http content encoding} , {http error status code} , {http headers} , {soap fault code} , {soap fault subcodes} , {soap headers} , {soap modules}
Binding Fault Reference	{soap modules}
Binding Message Reference	{http content encoding} , {http headers} , {soap headers} , {soap modules}
Binding Operation	{http content encoding default} , {http fault serialization} , {http input serialization} , {http location} , {http location ignore uncited} , {http method} , {http output serialization} , {http query parameter separator} , {soap action} , {soap mep} , {soap modules}
Endpoint	{http authentication realm} , {http authentication scheme}

HTTP Header	{ name }, { parent }, { required }, { type definition }
Interface Operation	{ rpc signature }, { safe }
SOAP Header Block	{ element declaration }, { mustUnderstand }, { parent }, { required }
SOAP Module	{ parent }, { ref }, { required }
Property	Where Defined
element declaration	SOAP Header Block.{ element declaration }
http authentication realm	Endpoint.{ http authentication realm }
http authentication scheme	Endpoint.{ http authentication scheme }
http content encoding	Binding Fault.{ http content encoding }, Binding Message Reference.{ http content encoding }
http content encoding default	Binding.{ http content encoding default }, Binding Operation.{ http content encoding default }
http cookies	Binding.{ http cookies }
http error status code	Binding Fault.{ http error status code }
http fault serialization	Binding Operation.{ http fault serialization }
http headers	Binding Fault.{ http headers }, Binding Message Reference.{ http headers }
http input serialization	Binding Operation.{ http input serialization }
http location	Binding Operation.{ http location }
http location ignore uncited	Binding Operation.{ http location ignore uncited }
http method	Binding Operation.{ http method }
http method default	Binding.{ http method default }
http output serialization	Binding Operation.{ http output serialization }
http query parameter separator	Binding Operation.{ http query parameter separator }
http query parameter separator default	Binding.{ http query parameter separator default }
mustUnderstand	SOAP Header Block.{ mustUnderstand }
name	HTTP Header.{ name }
parent	HTTP Header.{ parent }, SOAP Header Block.{ parent }, SOAP Module.{ parent }
ref	SOAP Module.{ ref }
required	HTTP Header.{ required }, SOAP Header Block.{ required }, SOAP Module.{ required }
rpc signature	Interface Operation.{ rpc signature }
safe	Interface Operation.{ safe }
soap action	Binding Operation.{ soap action }
soap fault code	Binding Fault.{ soap fault code }
soap fault	Binding Fault.{ soap fault subcodes }

subcodes	
soap headers	Binding Fault.{ soap headers }, Binding Message Reference.{ soap headers }
soap mep	Binding Operation.{ soap mep }
soap mep default	Binding.{ soap mep default }
soap modules	Binding.{ soap modules }, Binding Fault.{ soap modules }, Binding Fault Reference.{ soap modules }, Binding Message Reference.{ soap modules }, Binding Operation.{ soap modules }
soap underlying protocol	Binding.{ soap underlying protocol }
soap version	Binding.{ soap version }
type definition	HTTP Header.{ type definition }

C. Assertion Summary (Non-Normative)

This appendix summarizes assertions about WSDL 2.0 documents and components that are not enforced by the WSDL 2.0 schema. Each assertion is assigned a unique identifier which WSDL 2.0 processors may use to report errors.

Table C-1. Summary of Assertions about WSDL 2.0 Documents

Id	Assertion
OperationSafety-2028	An OPTIONAL <i>safe attribute information item</i> with the following Infoset properties:
WRPC-2050	Additionally, each even-numbered item (0, 2, 4, ...) in the list MUST be of type <i>xs:QName</i> and each odd-numbered item (1, 3, 5, ...) in the list MUST be of the subtype of <i>xs:token</i> described in the previous paragraph.

Table C-2. Summary of Assertions about WSDL 2.0 Components

Id	Assertion
FaultPropagationModification-2005	However, extensions or binding extensions MAY modify these rulesets.
HTTPAccessAuthentication-2127	If the { http authentication scheme } property is present, then this property MUST be present.
HTTPBinding-2083	When formulating the HTTP message to be transmitted, the HTTP request method used MUST be selected using one of the following:
HTTPBinding-2084	When formulating the HTTP message to be transmitted, content encoding for a given Binding Message Reference component is determined as follows:
HTTPBinding-2085	When formulating the HTTP fault message to be transmitted, content encoding for a given Binding Fault component is determined as follows:
HTTPBinding-2086	When formulating the HTTP message to be transmitted, the contents of the payload (i.e. the contents of the HTTP message body) MUST be what is defined by the corresponding Interface Message Reference or Interface Fault components, serialized as specified by the serialization format used.
HTTPBinding-2087	If the value is "#none", then the payload MUST be empty and the value of the corresponding serialization property ({ http input serialization } or { http output serialization }) is ignored.
HTTPBinding-2088	If the Interface Message Reference component or the Interface Fault component is declared using a non-XML type system (as considered in the Types section of [WSDL 2.0 Core Language]), then additional binding rules MUST be defined in

	an extension specification to indicate how to map those components into the HTTP envelope.
HTTPBinding-2089	The serialization rules for messages whose {message content model} is either "#element" or "#any", AND the serialization rules for fault messages, are as follows:
HTTPBindingFault-2105	The fault definition SHOULD agree with the definition of the HTTP error codes, as specified in section 8 of [IETF RFC 3205] .
HTTPBindingFault-2106	An integer value of this property identifies the error Status-Code as defined by [IETF RFC 2616] that the service will use in case the fault is returned.
HTTPBindingOperation-2093	When formulating the HTTP Request, the HTTP Request IRI is an absolute IRI reference and is the value of the {http location} property of the Binding Operation component, resolved using the value of the {address} property of the Endpoint component (see section 5 of [IETF RFC 3986]).
HTTPBindingOperation-2094	The first one is transmitted using an HTTP request, and the second one is transmitted using the corresponding HTTP response.
HTTPBindingOperation-2095	In cases where only one single message is being sent, the message body of the HTTP response MUST be empty.
HTTPBindingOperation-2098	It MUST contain an IRI reference and MUST NOT include a fragment identifier component.
HTTPBindingOperation-2100	The value of the serialization format used for a message is a media type which MUST be covered by this range.
HTTPBindingOperation-2101	Wild cards (for example, "application/*") SHOULD NOT be used in this <i>attribute information item</i> since they may lead to interoperability problems.
HTTPCookies-2126	A value of "true" means that the service relies on cookies and that the client MUST understand them.
HTTPHeader-2090	If the {http headers} property as defined in section 6.6 Declaring HTTP Headers exists and is not empty in a Binding Message Reference or Binding Fault component, HTTP headers conforming to each HTTP Header component contained in this {http headers} property MAY be serialized as follows:
HTTPHeader-2091	The HTTP binding MUST NOT set an HTTP header field corresponding to the value of the {name} property already set by another mechanism, such as the HTTP stack or another feature.
HTTPHeader-2092	If the value of an HTTP Header component's {required} property is "true", the inclusion of this HTTP header field is REQUIRED
HTTPHeader-2102	A Binding Message Reference or a Binding Fault component's {http headers} property MUST NOT contain multiple HTTP Header components with the same {name} property.
HTTPHeader-2103	This type MUST be a simple type.
HTTPHeader-2104	If the value is "true", then the HTTP header field MUST be included in the message.
HTTPQueryString-2115	The instance data MUST NOT contain elements with an <code>xs:nil</code> attribute whose value is "true".
HTTPQueryString-2116	When serializing an HTTP request that does not allow an HTTP message body, and when {http location ignore uncited} is "true", any element NOT cited in the {http location} property MUST be defined in the schema as <code>nillable</code> , or have a default value, or appear no less frequently than specified by the

	minOccurs value. The element declaration SHOULD NOT combine a default value with <code>nillable</code> .
HTTPSerialization-2099	The value of the <code>{http input serialization}</code> , <code>{http output serialization}</code> and <code>{http fault serialization}</code> properties is similar to the value allowed for the <code>Accept</code> HTTP header defined by the HTTP 1.1 specification, Section 14.1 (see [IETF RFC 2616]) and MUST follow the production rules defined in that section except for the following:
HTTPSerialization-2106	The <code>{http location}</code> property MUST conform to the following EBNF [ISO/IEC 14977:1996] grammar, which represents the patterns for constructing the request IRI:
HTTPSerialization-2107	If the <code>{style}</code> property of the Interface Operation bound has a value of " http://www.w3.org/ns/wsdli/style/iri " as defined in 4.2 IRI Style , and if the <code>{http location}</code> property of the Binding Operation component is present, the value of the <code>{http location}</code> property component is used as a template
HTTPSerialization-2108	The resulting IRI MUST be mapped to an URI for use in the HTTP Request as per section 3.1 "Mapping of IRIs to URIs" of the IRI specification [IETF RFC 3987].
HTTPSerialization-2109	The local name in a template SHOULD match at least one element from the instance data of the input message.
HTTPSerialization-2111	If this format is used then the <code>{style}</code> property of Interface Operation component being bound MUST contain a value of " http://www.w3.org/ns/wsdli/style/iri " as defined in 4.2 IRI Style , i.e. this serialization format may only be used to serialize the HTTP request corresponding to the initial message of an interface operation.
HTTPSerialization-2112	For the HTTP binding defined in this section (6. WSDL HTTP Binding Extension), " <code>application/x-www-form-urlencoded</code> " MAY be used as a serialization format for an input message (HTTP Request), but MUST NOT be used as a serialization format for an output or fault message (HTTP Response).
HTTPSerialization-2113	If not all elements from the instance data are cited in the <code>{http location}</code> property, or if the property is not present on the Binding Operation component, then additional serialization rules apply.
HTTPSerialization-2114	For elements of the instance data not cited in the <code>{http location}</code> property, a query string is constructed as follows.
HTTPSerialization-2117	If the HTTP request method used does not allow HTTP message body (e.g. "GET" and "DELETE"), and if the value of the <code>{http location ignore uncited}</code> property is "false", then the following rules apply.
HTTPSerialization-2118	If the HTTP request method used does allow an HTTP message body (e.g. "POST" and "PUT"), then the following rules apply.
HTTPSerialization-2119	The Content-Type HTTP header field must have the value <code>application/x-www-form-urlencoded</code> .
HTTPSerialization-2120	The Content-Type HTTP header MUST have the value <code>application/xml</code> [IETF RFC 3023], or a media type compatible with <code>application/xml</code> as specified in section 6.4.3.1 Serialization rules for XML messages .
HTTPSerialization-2121	this serialization format may only be used to serialize the HTTP request corresponding to the initial message of an interface operation.
HTTPSerialization-2122	Specifically, for the HTTP binding defined in this section (6. WSDL HTTP Binding Extension), " <code>multipart/form-data</code> " MAY be used as a serialization format for an input message (HTTP

	Request), but MUST NOT be used as a serialization format for an output or fault message (HTTP Response).
HTTPSerialization-2123	The Content-Disposition header MUST have the value form-data, and its name parameter is the local name of the element.
HTTPSerialization-2124	The Content-Type header MUST have the value:
HTTPSerialization-2125	The instance data MUST NOT contain elements with an xs:nil attribute whose value is "true".
IRIStyle-2051	When using this style, the value of the { message content model } property of the Interface Message Reference component corresponding to the initial message of the message exchange pattern MUST be "#element".
IRIStyle-2052	The sequence MUST only contain elements.
IRIStyle-2053	The sequence MUST contain only local element children.
IRIStyle-2054	The localPart of the element's QName MUST be the same as the Interface Operation component's { name }.
IRIStyle-2055	The complex type that defines the body of the element or its children elements MUST NOT contain any attributes.
IRIStyle-2056	The children elements of the sequence MUST derive from xs:simpleType, and MUST NOT be of the type or derive from xs:QName, xs:NOTATION, xs:hexBinary Or xs:base64Binary.
InOnlyComposition-2012	The in-only message exchange pattern consists of exactly one message as follows:
InOutComposition-2015	The in-out message exchange pattern consists of exactly two messages, in order, as follows:
InterfaceOperation-2096	202 when the MEP is "http://www.w3.org/ns/wsdli/in-only"
InterfaceOperation-2097	204 when the MEP is "http://www.w3.org/ns/wsdli/robust-in-only"
MultipartStyle-2057	When using this style, the value of the { message content model } property of the Interface Message Reference component corresponding to the initial message of the message exchange pattern MUST be "#element".
MultipartStyle-2058	The sequence MUST only contain elements.
MultipartStyle-2059	The sequence MUST contain only local element children.
MultipartStyle-2060	The attributes minOccurs and maxOccurs for these child elements MUST have a value 1.
MultipartStyle-2061	The localPart of the element's QName MUST be the same as the Interface Operation component's { name }.
MultipartStyle-2062	The complex type that defines the body of the element or its children elements MUST NOT contain any attributes.
MultipartStyle-2063	The sequence MUST NOT contain multiple children element declared with the same local name.
OperationSafety-2027	However, an operation SHOULD be marked safe if it meets the criteria for a safe interaction defined in Section 3.4 of [Web Architecture].
RPCStyle-2029	If the RPC style is used by an Interface Operation component then its { message exchange pattern } property MUST have the value either "http://www.w3.org/ns/wsdli/in-only" or "http://www.w3.org/ns/wsdli/in-out".
RPCStyle-2030	The value of the { message content model } property for the Interface Message Reference components of the { interface message references } property MUST be "#element".
RPCStyle-2031	The content model of input and output { element declaration } elements MUST be defined using a complex type that contains a sequence from XML Schema.

RPCStyle-2032	The input sequence MUST only contain elements and element wildcards.
RPCStyle-2033	The input sequence MUST NOT contain more than one element wildcard.
RPCStyle-2034	The element wildcard, if present, MUST appear after any elements.
RPCStyle-2035	The output sequence MUST only contain elements.
RPCStyle-2036	Both the input and output sequences MUST contain only local element children.
RPCStyle-2037	The local name of input element's QName MUST be the same as the Interface Operation component's name.
RPCStyle-2038	Input and output elements MUST both be in the same namespace.
RPCStyle-2039	The complex type that defines the body of an input or an output element MUST NOT contain any local attributes.
RPCStyle-2040	If elements with the same qualified name appear as children of both the input and output elements, then they MUST both be declared using the same named type.
RPCStyle-2041	The input or output sequence MUST NOT contain multiple children elements declared with the same name.
RobustInOnlyComposition-2013	The robust-in-only message exchange pattern consists of exactly one message as follows:
SOAPAction-2075	A <i>xs:anyURI</i> , which is an absolute IRI as defined by [IETF RFC 3987], to the Binding Operation component.
SOAPBinding-2065	When formulating the SOAP envelope to be transmitted, the contents of the payload (i.e., the contents of the SOAP Body <i>element information item</i> of the SOAP envelope) MUST be what is defined by the corresponding Interface Message Reference component.
SOAPBinding-2068	If the Interface Message Reference component is declared using a non-XML type system (as considered in the Types section of [WSDL 2.0 Core Language]), then additional binding rules MUST be defined to indicate how to map those components into the SOAP envelope.
SOAPBinding-2069	Every SOAP binding MUST indicate what version of SOAP is in use for the operations of the interface that this binding applies to.
SOAPBinding-2070	Every SOAP binding MUST indicate what underlying protocol is in use.
SOAPBindingFault-2071	For every Interface Fault component contained in an Interface component, a mapping to a SOAP Fault MUST be described.
SOAPBindingFault-2072	when the value of the <code>{soap_version}</code> is "1.2", the allowed QNames MUST be the ones defined by [SOAP 1.2 Part 1: Messaging Framework (Second Edition)], section 5.4.6
SOAPHTTPProperties-2064	These properties MUST NOT be used unless the underlying protocol is HTTP.
SOAPHTTPSelection-2082	This default binding rule is applicable when the value of the <code>{soap_underlying_protocol}</code> property of the Binding component is "http://www.w3.org/2003/05/soap/bindings/HTTP/". If the SOAP MEP selected as specified above has the value "http://www.w3.org/2003/05/soap/mep/request-response/" then the HTTP method used is "POST". If the SOAP MEP selected has the value "http://www.w3.org/2003/05/soap/mep/soap-response/" then the HTTP method used is "GET".
SOAPHeaderBlock-2077	When its value is "true", the SOAP header block MUST be

	decorated with a SOAP <i>mustUnderstand</i> <i>attribute information item</i> with a value of "true"; if so, the XML element declaration referenced by the { element declaration } property MUST allow this SOAP <i>mustUnderstand</i> <i>attribute information item</i> .
SOAPHeaderBlock-2078	If the value is "true", then the SOAP header block MUST be included in the message.
SOAPHeaderBlock-2079	The value of the <i>element attribute information item</i> MUST resolve to a global element declaration from the { element declarations } property of the Description component.
SOAPMEP-2074	A <i>xs:anyURI</i> , which is an absolute IRI as defined by [IETF RFC 3987], to the Binding Operation component.
SOAPMEPDefault-2073	A <i>xs:anyURI</i> , which is an absolute IRI as defined by [IETF RFC 3987], to the Binding component.
SOAPMEPSelection-2080	For a given Interface Operation component, if there is a Binding Operation component whose { interface operation } property matches the component in question and its { soap mep } property has a value, then the SOAP MEP is the value of the { soap mep } property. Otherwise, the SOAP MEP is the value of the Binding component's { soap mep default }, if any. Otherwise, the Interface Operation component's { message exchange pattern } property MUST have the value "http://www.w3.org/ns/wsdl/in-out", and the SOAP MEP is the URI "http://www.w3.org/2003/05/soap/mep/request-response/" identifying the SOAP Request-Response Message Exchange Pattern as defined in [SOAP 1.2 Part 2: Adjuncts (Second Edition)].
SOAPModule-2076	A <i>xs:anyURI</i> , which is an absolute IRI as defined by [IETF RFC 3987].
WRPC-2042	OPTIONAL, but MUST be present when the style is RPC
WRPC-2043	Values for the second component MUST be chosen among the following four: "#in", "#out", "#inout" "#return".
WRPC-2044	The value of the first component of each pair (<i>q</i> , <i>t</i>) MUST be unique within the list.
WRPC-2045	For each child element of the input and output messages of the operation, a pair (<i>q</i> , <i>t</i>), whose first component <i>q</i> is equal to the qualified name of that element, MUST be present in the list, with the caveat that elements that appear with cardinality greater than one MUST be treated as a single element.
WRPC-2046	For each pair (<i>q</i> , <i>#in</i>), there MUST be a child element of the input element with a name of <i>q</i> . There MUST NOT be a child element of the output element with the name of <i>q</i> .
WRPC-2047	For each pair (<i>q</i> , <i>#out</i>), there MUST be a child element of the output element with a name of <i>q</i> . There MUST NOT be a child element of the input element with the name of <i>q</i> .
WRPC-2048	For each pair (<i>q</i> , <i>#inout</i>), there MUST be a child element of the input element with a name of <i>q</i> . There MUST also be a child element of the output element with the name of <i>q</i> .
WRPC-2049	For each pair (<i>q</i> , <i>#return</i>), there MUST be a child element of the output element with a name of <i>q</i> . There MUST NOT be a child element of the input element with the name of <i>q</i> .

Table C-3. Summary of Assertions about Messages

Id	Assertion
HTTPSerialization-2110	Cited elements (i.e. elements referenced in templates) MUST NOT carry an <i>xs:nil</i> attribute whose value is "true"

SOAP12Binding-SOAPDetail-2081	If any, the value of the SOAP "Detail" element MUST be the <i>element information item</i> identified by the { element declaration } property of the Interface Fault component.
SOAPBinding-2066	If the value is "#none", then the payload MUST be empty.
SOAPBinding-2067	If the value is "#element", then the payload MUST be the <i>element information item</i> identified by the { element declaration } property of the Interface Message Reference component.

Table C-4. Summary of Assertions about Message Exchanges

Id	Assertion
FaultDelivery-2008	The fault message MUST be delivered to the same target node as the message it replaces, unless otherwise specified by an extension or binding extension. If there is no path to this node, the fault MUST be discarded.
FaultDelivery-2010	The fault message MUST be delivered to the originator of the triggering message, unless otherwise specified by an extension or binding extension. Any node MAY propagate a fault message, and MUST NOT do so more than once for each triggering message. If there is no path to the originator, the fault MUST be discarded.
FaultPropagation-2003	Nodes that generate faults MUST attempt to propagate the faults in accordance with the governing ruleset, but it is understood that any delivery of a network message is best effort, not guaranteed.
FaultPropagation-2004	When a fault is generated, the generating node MUST attempt to propagate the fault, and MUST do so in the direction and to the recipient specified by the ruleset.
FaultReplacesMessage-2007	When the Fault Replaces Message propagation rule is in effect, any message after the first in the pattern MAY be replaced with a fault message, which MUST have identical direction.
InOnlyFaults-2013	The in-only message exchange pattern uses the rule 2.2.3 No Faults propagation rule .
InOutFaults-2016	The in-out message exchange pattern uses the rule 2.2.1 Fault Replaces Message propagation rule .
MEPDescriptiveness-2002	by some prior agreement, another node and/or the service MAY send messages (to each other or to other nodes) that are not described by the pattern.
MEPTermination-2006	Generation of a fault, regardless of ruleset, terminates the exchange.
MessageTriggersFault-2009	When the Message Triggers Fault propagation rule is in effect, any message, including the first in the pattern, MAY trigger a fault message, which MUST have opposite direction.
NoFaults-2011	When the No Faults propagation rule is in effect, faults MUST NOT be propagated.
NodeIdentity-2001	A node MAY be accessible via more than one physical address or transport.
RobustInOnlyFaults-2014	The robust in-only message exchange pattern uses the rule 2.2.2 Message Triggers Fault propagation rule .